Formula 1 Podium Prediction — Milestone 1 Analytical Report

Team: (add team name and member names / IDs here)

This report is generated from the executed notebook `f1_podium_prediction.ipynb` : it contains the notebook's explanations, computed conclusions, figures and small result tables, without the code. Section numbers are the same as in the notebook. Large tables are in the notebook only.

Task. Predict, for every driver entering a race, whether they finish on the podium (`positionOrder <= 3`), using only information available **after qualifying and before the race start**.

Unit of analysis. One driver in one race — the key `(raceId, driverId)` is unique in the modeling table.

How to read this notebook. Every step follows the same pattern: *what we checked → what the data showed (table or plot) → what we decided and why*. Numbers that depend on a trained model are printed by the code itself, so the text always matches the run.

#	Section	Milestone item
1	Setup and raw data load	Raw files stay immutable
2	EDA before cleaning + data audit	EDA, PK/FK checks
3	Cleaning	Sentinels, typing, duplicate driver–race rows
4	EDA after cleaning	Impact of cleaning
5	Data-engineering questions 1–3	Multi-hop joins, answers, plots
6	Leakage audit and feature engineering	Leakage-safe form / standings / grid / qualifying features
7	Train / validation / test split	Split justification
8	Feature selection evidence	Plots, correlations, trials
9	Pre-processing and order experiments	Effect of each step, order experiments
10	Models	Baseline, statistical models, shallow NN, tuning
11	Model comparison	All metrics on train / validation / test
12	Feature-group ablation	Experiments
13	Explainability	Permutation importance, SHAP, LIME
14	Inference function	Complete and partially-missing record
15	Limitations and summary	Report material

---

1. Setup and raw data load

```
Data folder: Data
```

1.1 Load the raw tables exactly as they are on disk

Every CSV is read **as text**, with pandas' automatic missing-value detection turned off. This way the raw frames are a faithful copy of the files: nothing is silently converted, and we can count the placeholder values ourselves.

To prove the raw files are never modified, we store a SHA-256 fingerprint of each file now and compare it again at the very end of the notebook. All cleaning happens on copies (`CLEAN`).

2. EDA before cleaning and data audit

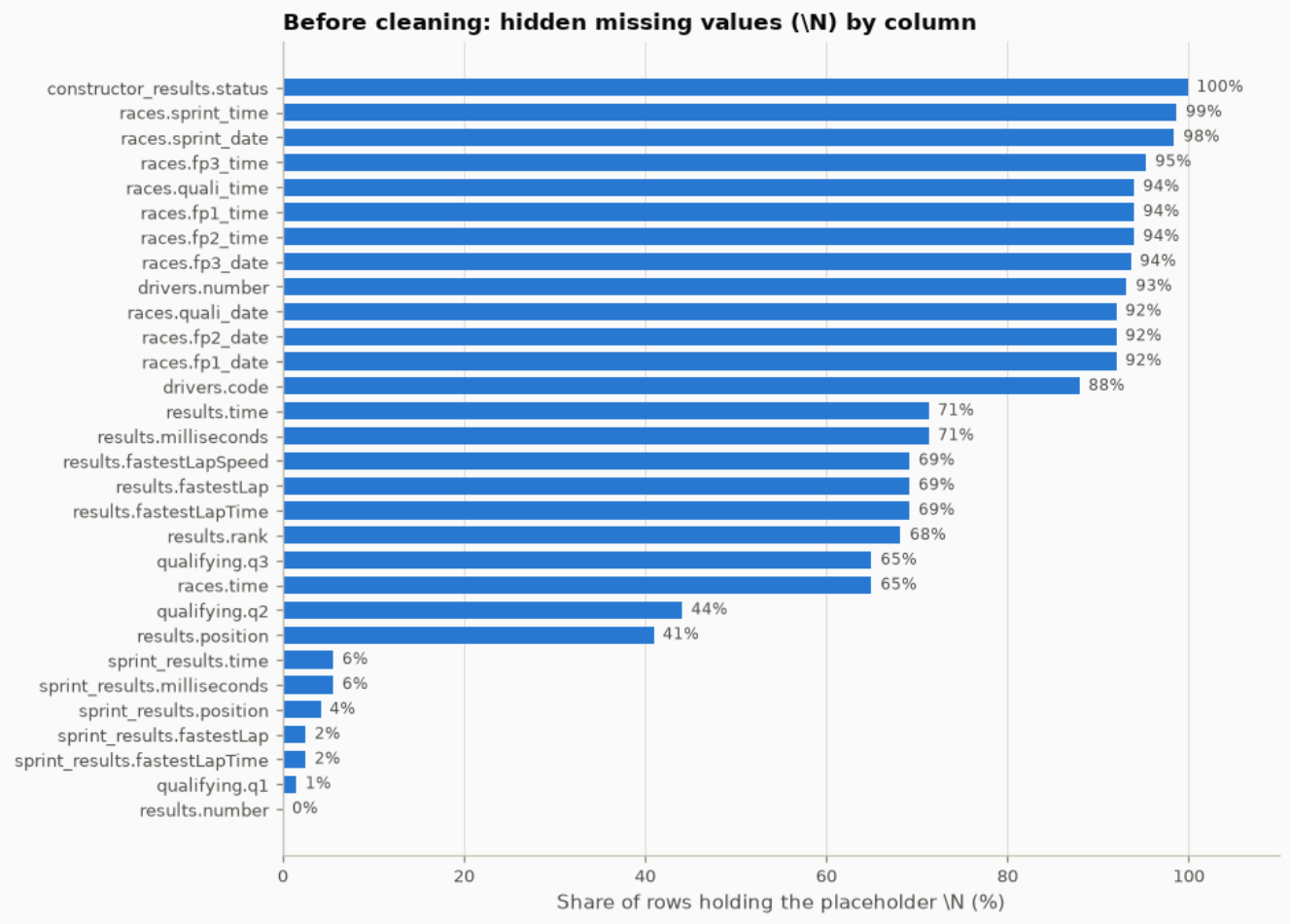
The audit is done in code, not by eye. Each check below produces a table or a plot, and each finding leads to one cleaning rule in Section 3.

2.1 Placeholder ("sentinel") missing values

The files use the literal text `\N` for "no value". We scan every cell of every table for `\N`, for empty strings, and for other common placeholders, so we do not assume `\N` is the only one.

```
Distinct placeholder kinds found: ["''", "'\\\\N'", 'leading/trailing space']
```

```
Default read: pandas sees 68 missing cells; the files really contain 160,144 placeholder cells.
Default read: 57 of 120 columns are typed as text.
```



What this shows.

- `\N` is the main placeholder, and `qualifying.q2` / `q3` also contain **empty strings** — a second placeholder that a `\N`-only rule would miss.
- A default read reports almost no missing data, while tens of thousands of cells are really empty. Any statistic computed on such a read would be wrong.
- The missingness is **structural, not random**: `results.position`, `time` and `milliseconds` are empty for drivers who did not finish; `fastestLap*` only exists from 2004; `q2`/`q3` are empty for drivers knocked out early; the practice and sprint timetable columns in `races` only exist for recent seasons. So we convert these to proper missing values and **do not fill them in** at this stage.
- One nationality value has a trailing space (`'Argentinian '`), which would create a false extra category.

2.2 Types and formats

Because of the placeholders, columns that hold numbers, dates and durations are all text. Before converting we check which **formats** actually occur, so the parser covers all of them.

What this shows. Durations come in several shapes: `M:SS.mmm` (lap and qualifying times), `SS.mmm` (pit stops under a minute), `M:SS.mmm` pit stops (long stops, usually under a red flag), `H:MM:SS.mmm` (winner's race time) and `+S.mmm` gaps for other finishers. One qualifying time has the shape `99:99.999` — a lap of more than ten minutes, which is not physically possible and is checked in Section 3. Dates are consistently `YYYY-MM-DD`.

2.3 Primary keys

A primary key must be unique and never empty. We test the declared key of every table.

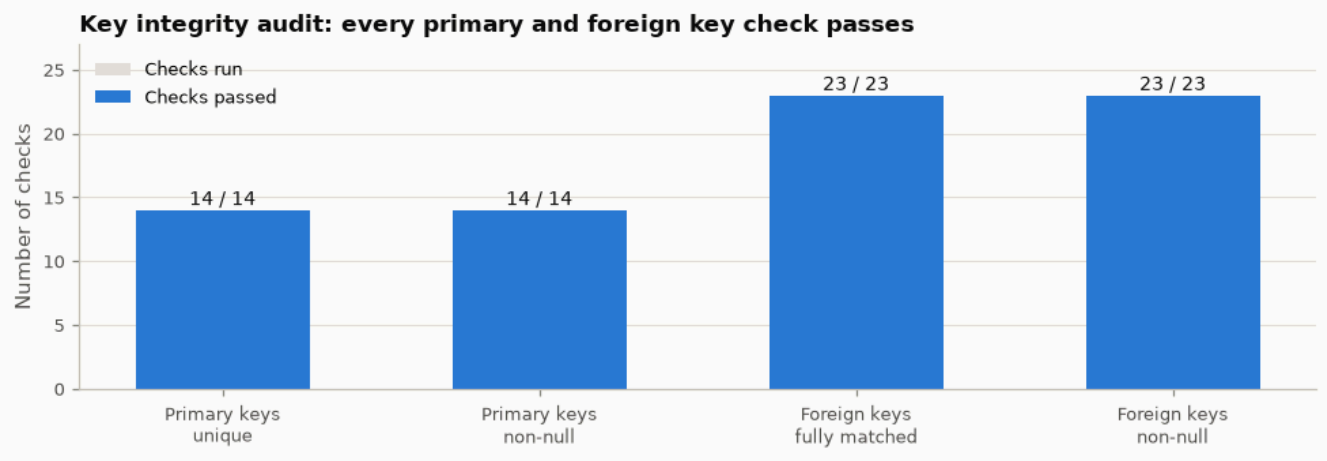
	table	primary key	rows	duplicate keys	null keys	status
0	circuits	circuitId	77	0	0	OK
1	constructors	constructorId	212	0	0	OK
2	drivers	driverId	861	0	0	OK
3	races	raceId	1125	0	0	OK
4	seasons	year	75	0	0	OK
5	status	statusId	139	0	0	OK
6	results	resultId	26759	0	0	OK
7	sprint_results	resultId	360	0	0	OK
8	qualifying	qualifyId	10494	0	0	OK
9	driver_standings	driverStandingsId	34863	0	0	OK
10	constructor_standings	constructorStandingsId	13391	0	0	OK
11	constructor_results	constructorResultsId	12625	0	0	OK
12	lap_times	raceId + driverId + lap	589081	0	0	OK
13	pit_stops	raceId + driverId + stop	11371	0	0	OK

2.4 Foreign keys

Every foreign key value must exist in the table it points to. An unmatched ("orphan") value would silently drop rows in an inner join or create empty rows in a left join.

	child	parent	rows	null	orphans	status
0	races.circuitId	circuits.circuitId	1125	0	0	OK
1	races.year	seasons.year	1125	0	0	OK
2	results.raceId	races.raceId	26759	0	0	OK
3	results.driverId	drivers.driverId	26759	0	0	OK
4	results.constructorId	constructors.constructorId	26759	0	0	OK
5	results.statusId	status.statusId	26759	0	0	OK
6	sprint_results.raceId	races.raceId	360	0	0	OK
7	sprint_results.driverId	drivers.driverId	360	0	0	OK
8	sprint_results.constructorId	constructors.constructorId	360	0	0	OK
9	sprint_results.statusId	status.statusId	360	0	0	OK
10	qualifying.raceId	races.raceId	10494	0	0	OK
11	qualifying.driverId	drivers.driverId	10494	0	0	OK
12	qualifying.constructorId	constructors.constructorId	10494	0	0	OK
13	driver_standings.raceId	races.raceId	34863	0	0	OK
14	driver_standings.driverId	drivers.driverId	34863	0	0	OK
15	constructor_standings.raceId	races.raceId	13391	0	0	OK
16	constructor_standings.constructorId	constructors.constructorId	13391	0	0	OK
17	constructor_results.raceId	races.raceId	12625	0	0	OK
18	constructor_results.constructorId	constructors.constructorId	12625	0	0	OK
19	lap_times.raceId	races.raceId	589081	0	0	OK
20	lap_times.driverId	drivers.driverId	589081	0	0	OK
21	pit_stops.raceId	races.raceId	11371	0	0	OK
22	pit_stops.driverId	drivers.driverId	11371	0	0	OK

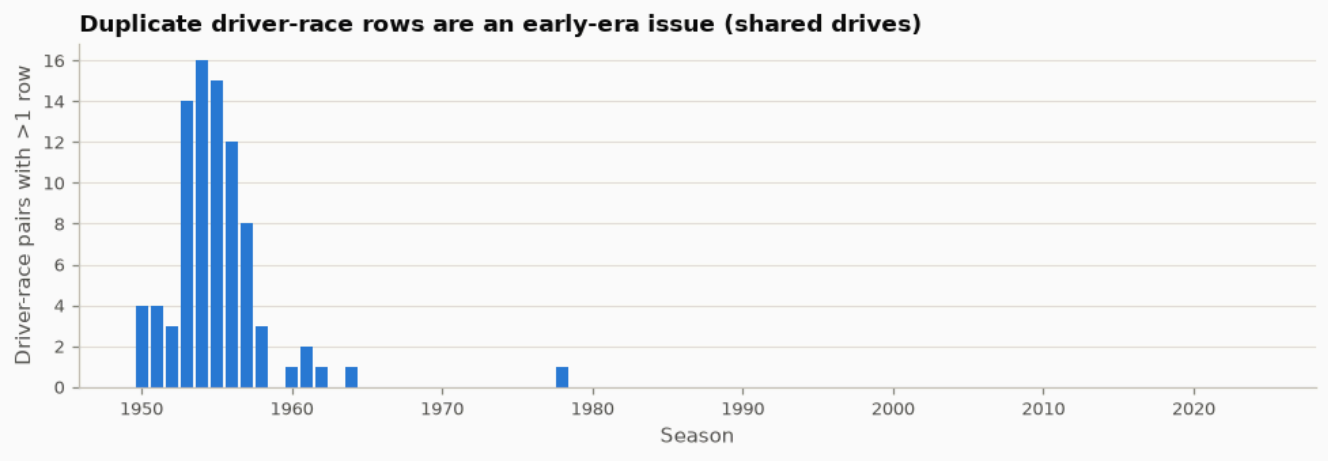
```
Qualifying entries with no race result row: 0 of 10494
```



What this shows. All primary keys are unique and non-null, and all foreign keys reconcile with zero orphans. This means joins between the tables will neither drop rows nor invent them, so no repair is needed here. The declared keys are sound — but the **modeling grain** `(raceId, driverId)` is *not* the primary key of `results` (that is `resultId`), so it needs its own check.

2.5 Is `(raceId, driverId)` unique in `results`?

```
176 result rows belong to 85 driver–race pairs that appear more than once (91 surplus rows).
Pairs where the entries differ in grid slot: 81, in constructor: 16
```



What this shows. The duplicates are concentrated in 1950–1964. In those seasons a driver could take over a team-mate's car mid-race, and each car the driver sat in is recorded as a separate result row (different car number, different grid slot, different laps). A few later cases (1962, 1978) are drivers entered by two teams who did not qualify. The rule for collapsing these rows is chosen in Section 3.4.

2.6 Other consistency checks found during exploration

Findings carried into cleaning.

#	Finding	Evidence	Cleaning rule (Section 3)
1	Two placeholders for missing: `\N` and empty string	2.1	Convert both to real missing values
2	Missing values are structural (DNF, era, knockout qualifying)	2.1	Keep as missing; do not fill
3	Numbers, dates and durations stored as text	2.2	Semantic typing; durations to seconds
4	One impossible qualifying time (over ten minutes)	2.2	Plausibility rule sets it to missing
5	Trailing space in a nationality; `Argentine` / `Argentinian`; `USA` / `United States`	2.1, 2.6	Trim text; harmonise the two spellings
6	Driver–race pairs duplicated by shared drives	2.5	Collapse to one row per driver–race
7	`grid = 0` means "no grid slot" (pit-lane start or did not start), not "ahead of pole"	2.6	Keep in cleaning; handled as a feature in Section 6
8	From 1965 on, every race has exactly 3 rows with `positionOrder <= 3`. Up to 1964, two drivers who shared a car share one `positionOrder`, so some races have 4–5 podium rows	2.6	`positionOrder` is a safe target source; the early ties are genuine shared podiums and are kept

---

3. Cleaning

All cleaning is applied to copies; `RAW` is left untouched.

3.1 Sentinels and whitespace

3.2 Semantic typing

- **Durations → seconds (float).** A single parser handles `SS.mmm`, `M:SS.mmm` and `H:MM:SS.mmm`. Gap strings such as `+5.478` in `results.time` are not durations of the race; the numeric race time is already in `results.milliseconds`, so we derive `race_time_s` from that and keep `time` as text.
- **Dates → datetime.** Race date and time are also combined into one timestamp.
- **Numbers.** Every remaining column is converted to a number **only if every non-missing value converts**, so the conversion can never create new missing values. Whole-number columns use the nullable integer type, which keeps IDs as integers even when some rows are missing.

to	datetime	float	integer	seconds (float)
table				
circuits	0	2	2	0
constructor_results	0	1	3	0
constructor_standings	0	1	5	0
constructors	0	0	1	0
driver_standings	0	1	5	0
drivers	1	0	2	0
lap_times	0	0	5	1
pit_stops	0	0	5	1
qualifying	0	0	6	3
races	6	0	4	0
results	0	2	13	1
seasons	0	0	1	0
sprint_results	0	0	13	1
status	0	0	1	0

```
Largest disagreement between parsed lap time and the milliseconds column: 0.000000 s
```

The duration parser agrees with the independent `milliseconds` column on every one of the ~589k laps, which validates it before we rely on it for qualifying times (which have no such second column).

3.3 Value-level fixes

Three issues found in the audit are fixed with explicit, testable rules:

1. **Unrepresentative qualifying times.** A qualifying time slower than 1.5× the fastest lap of the same session is not a real flying lap (a typo, or a slow lap recorded after the driver's fast laps were deleted). Such values are set to missing rather than dropping the row, so the driver's qualifying *position* is kept.
2. **Category spelling.** `United States` → `USA` and `Argentinian` → `Argentine`, so one country is one category.
3. **Pit-stop durations inflated by red flags.** Stops longer than five minutes are cars waiting in the pit lane during a stoppage, not service time. We keep the row and add a flag, because pit stops are not used as features (they happen after the cutoff) and deleting them would change the stop count.

	raceId	driverId	position	q1	q1_s	session_best_s
2567	255	87	24	16:42.640	1002.64	98.023

	raceId	driverId	position	q2	q2_s	session_best_s
9749	1107	815	15	2:06.688	126.688	64.951

```
Qualifying times set to missing by the plausibility rule: 2
Pit stops flagged as red-flag waits (> 5 min): 478
Driver nationalities after harmonising: 42 (was 43 )
```

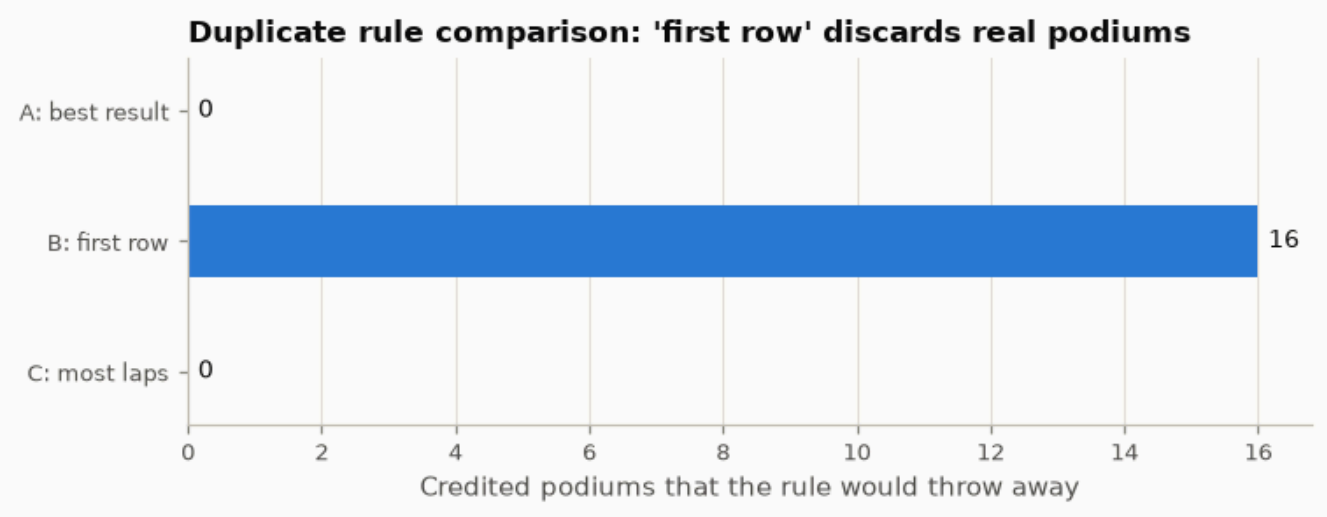
3.4 One row per driver per race

Each row of the modeling table is one prediction, so each driver must appear once per race. We compare three candidate rules on the duplicated pairs before choosing.

Rule	Idea	Weakness
A. Keep the **best classified result** (lowest `positionOrder`)	The target asks "did this driver finish on the podium?" A driver who was credited with a podium in any car did.	Chooses the row using the outcome
B. Keep the **first recorded row** (lowest `resultId`)	Simple, does not look at the outcome	Recording order is arbitrary; can discard a real podium
C. Keep the row with the **most laps**	The car the driver spent most time in	Also uses post-race information; can discard a podium

	duplicated pairs	pairs with a credited podium	podiums kept	podiums lost
A: best result	85	22	22	0
B: first row	85	22	6	16
C: most laps	85	22	22	0

```
Pairs where rules A and C keep a different row: 0
```



Decision: rule A — keep the row with the lowest `positionOrder` (ties broken by lowest `resultId`).

- Rule B is rejected on evidence: it would label a driver "no podium" in 16 races where the driver was officially credited with one.
- Rules A and C keep every podium and, on this data, select exactly the same rows. We state the rule as A because it is defined directly by the target column, so it is guaranteed to preserve the label even on future data, whereas C preserves it only by coincidence.
- It is deterministic: the tie-break on `resultId` means a re-run always keeps the same row.
- We also keep two extra columns, `n_entries` and `shared_drive`, so the collapsed rows stay identifiable.

Known cost. Rule A uses the outcome to decide which row survives, so the `grid` value of a collapsed row may belong to the car the driver took over rather than the one they started in. This matters only for those few dozen rows, all in 1950–1978. The modeling table starts in 2003 (Section 6), so no modeling row is affected; the rule influences only long-history statistics in the data-engineering questions.

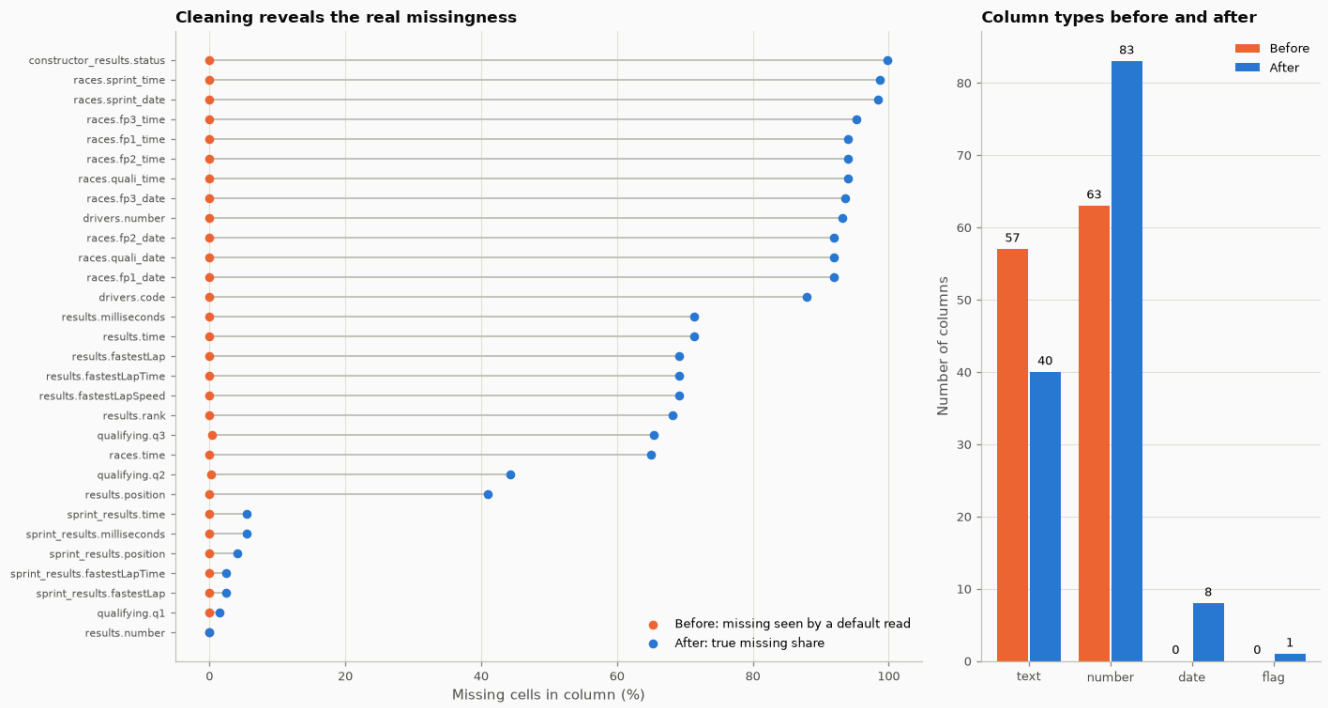
```
results: 26,759 rows -> 26,668 rows  (91 surplus rows removed)
Rows marked as shared_drive: 85
```

---

4. EDA after cleaning

This section repeats the before-cleaning views on the cleaned copies, to show what each cleaning step changed, and then explores the cleaned data.

4.1 Impact of cleaning

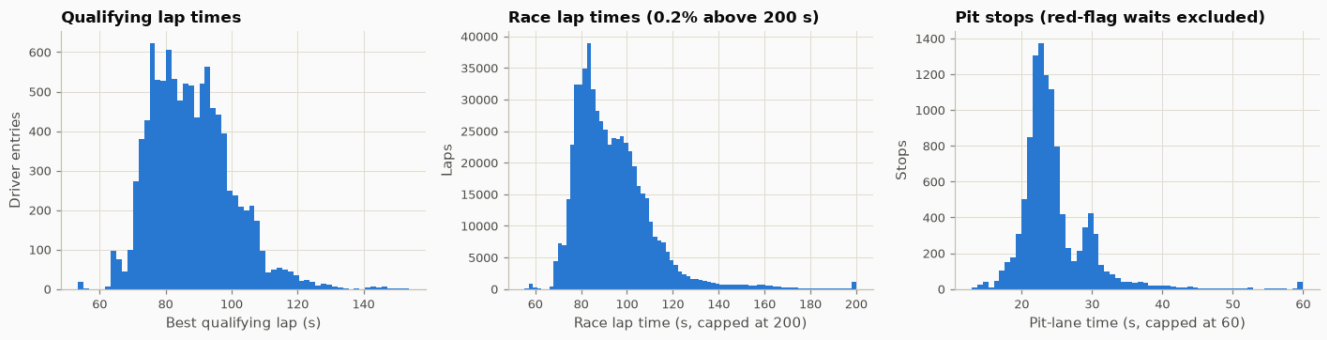


	before	after
missing cells visible to pandas	68.0	160144.0
columns typed as text	57.0	40.0
duplicate (raceId, driverId) rows in results	91.0	0.0
nationality categories	43.0	42.0
circuit countries	35.0	34.0
max qualifying Q1 time (s)	NaN	153.9

Impact of cleaning. The true missingness is now visible to pandas (orange dots at zero move to their real position in blue), the text-typed columns that held numbers, dates and durations are now usable types, the grain is unique, and two false categories are gone. The maximum Q1 time could not even be computed before cleaning, because the column was text.

4.2 Distributions that only exist after typing

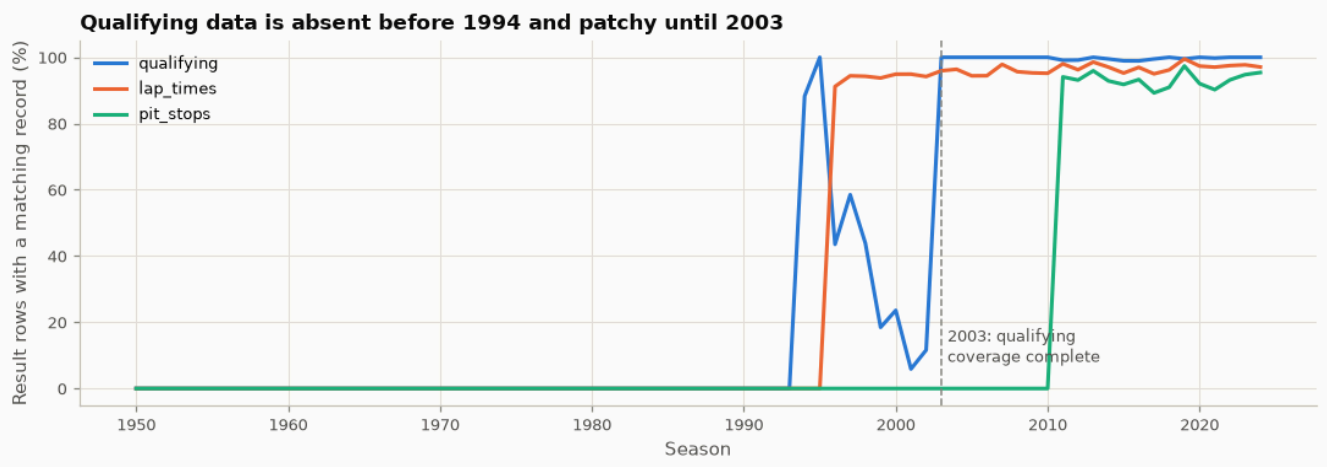
Durations were text before cleaning, so they could not be plotted at all. After conversion we can inspect them and confirm the values are plausible.



Qualifying laps fall between roughly 55 s and 130 s, depending on the circuit length. Race laps have a long right tail from safety-car and in/out laps. Pit-lane times centre around 20–25 s. None of these contain impossible values after cleaning.

4.3 Coverage over time: which tables exist for which seasons?

This decides which seasons can support qualifying-based features.

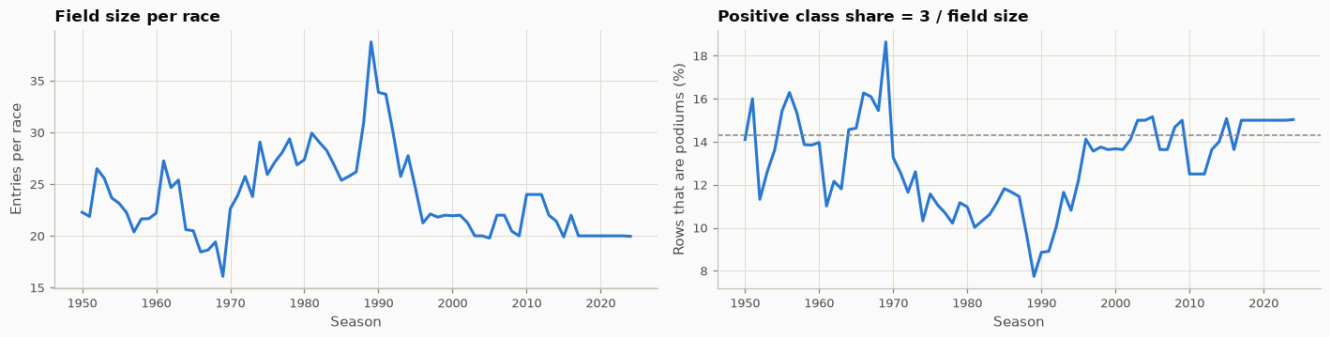


year	1993	1994	1996	1999	2002	2003	2010	2024
qualifying	0.0	88.3	43.5	18.5	11.6	100.0	100.0	100.0

```
First season from which qualifying coverage stays at or above 95%: 2003
```

What this shows. The `qualifying` table has no rows before 1994 and covers only a fraction of entries from 1996 to 2002. From **2003** onward it covers essentially every entry. Because the prediction cutoff is "after qualifying", qualifying information is central to the task — so the modeling table is restricted to **2003–2024**. Earlier seasons are still used as *history* when computing form features (a driver racing in 2003 has a record from before 2003).

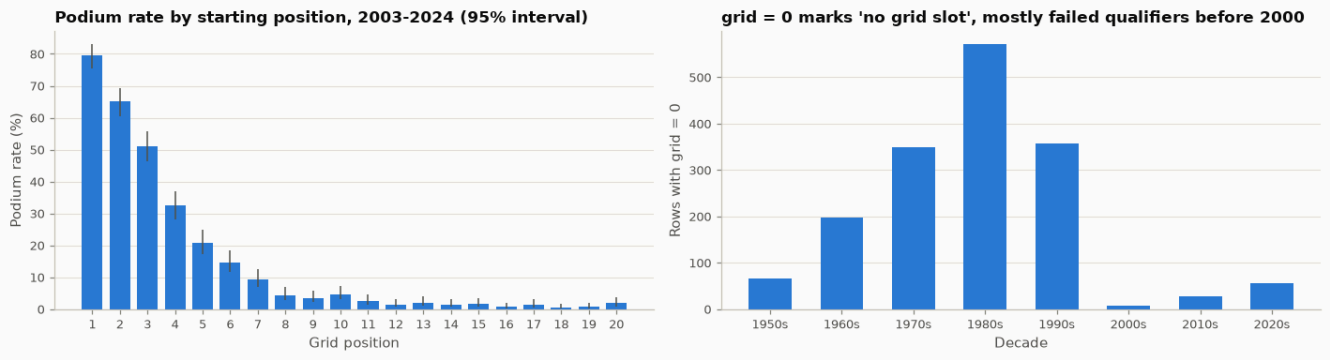
4.4 Target and field size




```
2003–2024: 8,970 driver-race rows, 428 races, podium share 14.3%
```

What this shows. There are exactly three podium places per race, so the share of positive rows is simply 3 divided by the field size. With 20–22 cars it is about **14–15%**. The classes are imbalanced, which is why accuracy is a poor metric here (always predicting "no podium" would already be about 85% accurate) and why precision–recall based metrics are preferred later.

4.5 Starting position and the target




```
Podium rate of grid = 0 rows, 2003–2024: 0.0%  (n = 86)
```

What this shows.

- Podium probability falls steeply and smoothly with grid position: pole converts about three times in four, and from tenth place back it is a rare event. Grid position will be the strongest single predictor, and the relationship is non-linear.
- `grid = 0` does **not** mean "ahead of pole". Before 2000 it mostly marks drivers who failed to qualify; in recent seasons it marks pit-lane starts. Treating 0 as a number would tell a model these are the best starters, so Section 6 recodes it.

---

5. Data-engineering questions

Each answer states the **joins**, the **grain**, how **missing data** was handled and whether the **pre-race cutoff** matters, followed by the result and a plot. All three questions describe the past using race outcomes, so they are descriptive analyses — the cutoff applies to *features for prediction*, not to these summaries.

One shared base table is used: de-duplicated `results` joined to `races` (season, round, circuit, date). Its grain is one row per driver per race.

```
Base fact table: 26,668 rows, one per driver per race; joins added no rows.
```

Question 1 — Which circuits convert a front-row start into a podium most often, and does the ranking change in the hybrid era?

Joins. `results → races → circuits` (three tables) for the circuit of each result, plus `results → qualifying` on `(raceId, driverId)` for the qualifying position.

Qualifying position vs. grid. These are two different things:

- `qualifying.position` — where the driver finished the qualifying session.
- `results.grid` — where the driver actually started, after penalties, and on sprint weekends in 2021–2022 after the sprint.

A "front-row start" is defined on the **grid** (`grid` is 1 or 2), because that is the position the car really started from. The qualifying-based version is reported next to it to show how much the two differ.

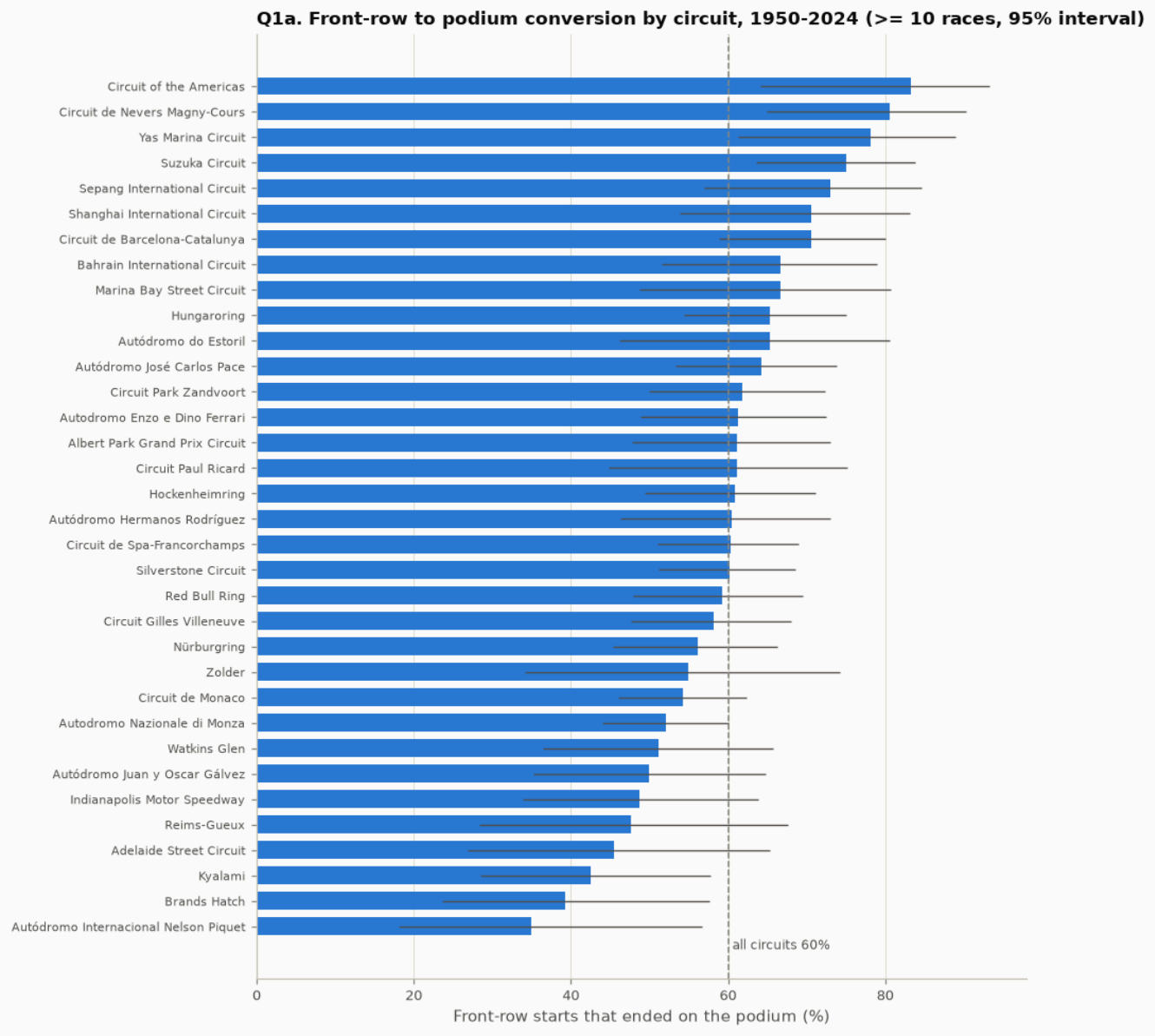
Grain. One row per front-row starter per race; aggregated to circuit × era. Each race contributes two rows.

Missingness. `grid` is never missing. `qualifying.position` only has full coverage from 2003, so everything that uses it is limited to 2003–2024.

Small samples. A circuit with three races has six front-row starts, so its rate is mostly noise. We require a minimum number of races and show 95% Wilson intervals.

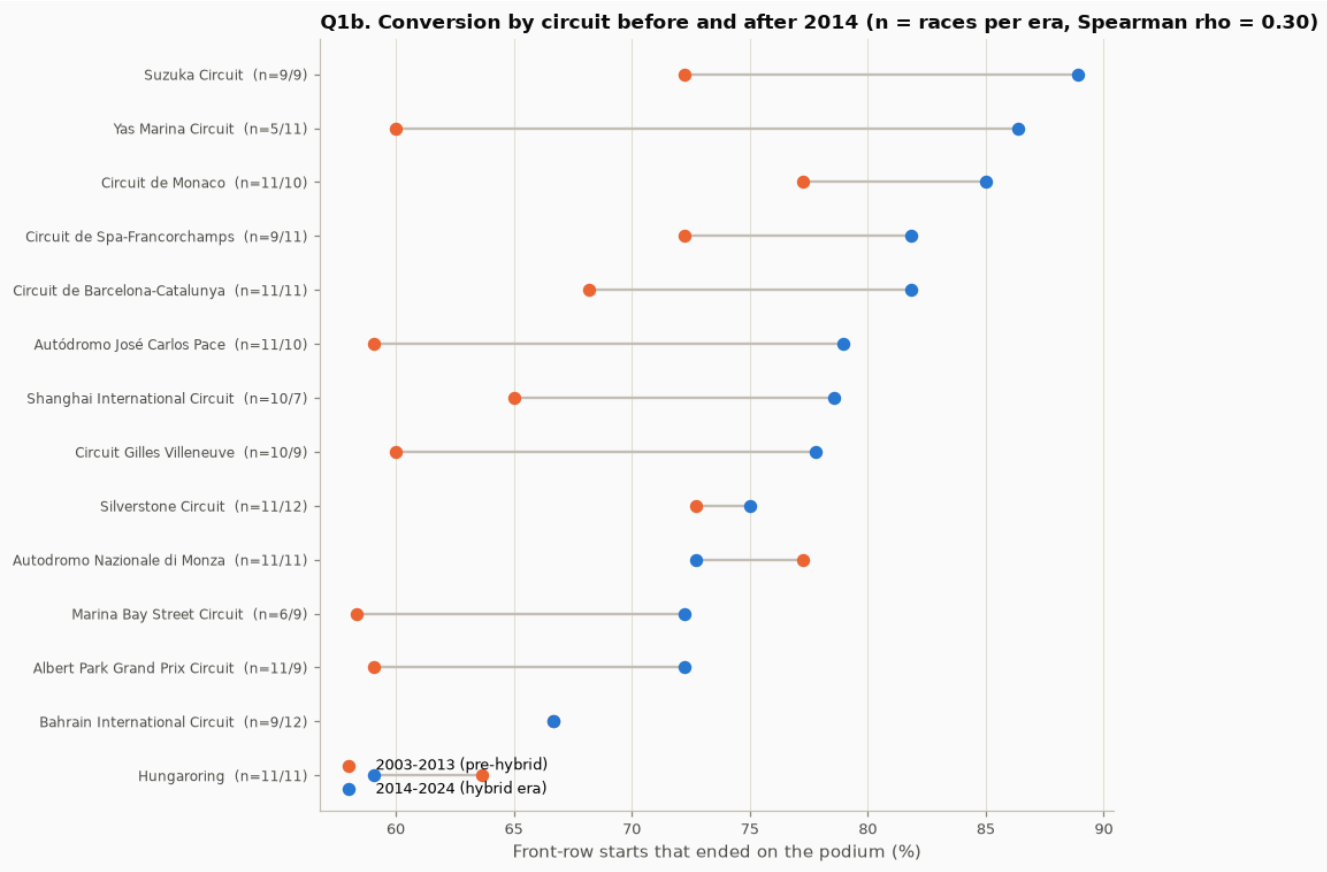
```
All seasons: front-row starters reach the podium in 60.1% of starts (34 circuits with >= 10 races).
```

	races	starts	podiums	rate	lo	hi
circuit						
Circuit of the Americas	12	24	20	83.3	64.1	93.3
Circuit de Nevers Magny-Cours	18	36	29	80.6	65.0	90.2
Yas Marina Circuit	16	32	25	78.1	61.2	89.0
Suzuka Circuit	34	68	51	75.0	63.6	83.8
Sepang International Circuit	19	37	27	73.0	57.0	84.6
Shanghai International Circuit	17	34	24	70.6	53.8	83.2
Circuit de Barcelona-Catalunya	34	68	48	70.6	58.9	80.1
Bahrain International Circuit	21	42	28	66.7	51.6	79.0
Reims-Gueux	11	21	10	47.6	28.3	67.6
Adelaide Street Circuit	11	22	10	45.5	26.9	65.3
Kyalami	20	40	17	42.5	28.5	57.8
Brands Hatch	14	28	11	39.3	23.6	57.6
Autódromo Internacional Nelson Piquet	10	20	7	35.0	18.1	56.7


Q1a. Front-row to podium conversion by circuit, 1950-2024 (>= 10 races, 95% interval)
Circuit of the Americas
Circuit de Nevers Magny-Cours
Yas Marina Circuit
Suzuka Circuit
Sepang International Circuit
Shanghai International Circuit
Circuit de Barcelona-Catalunya
Bahrain International Circuit
Marina Bay Street Circuit
Hungaroring
Autódromo do Estoril
Autódromo José Carlos Pace
Circuit Park Zandvoort
Autodromo Enzo e Dino Ferrari
Albert Park Grand Prix Circuit
Circuit Paul Ricard
Hockenheimring
Autódromo Hermanos Rodríguez
Circuit de Spa-Francorchamps
Silverstone Circuit
Red Bull Ring
Circuit Gilles Villeneuve
Nürburgring
Zolder
Circuit de Monaco
Autodromo Nazionale di Monza
Watkins Glen
Autódromo Juan y Oscar Gálvez
Indianapolis Motor Speedway
Reims-Gueux
Adelaide Street Circuit
Kyalami
Brands Hatch
Autódromo Internacional Nelson Piquet
all circuits 60%
0
20
40
60
80
Front-row starts that ended on the podium (%)

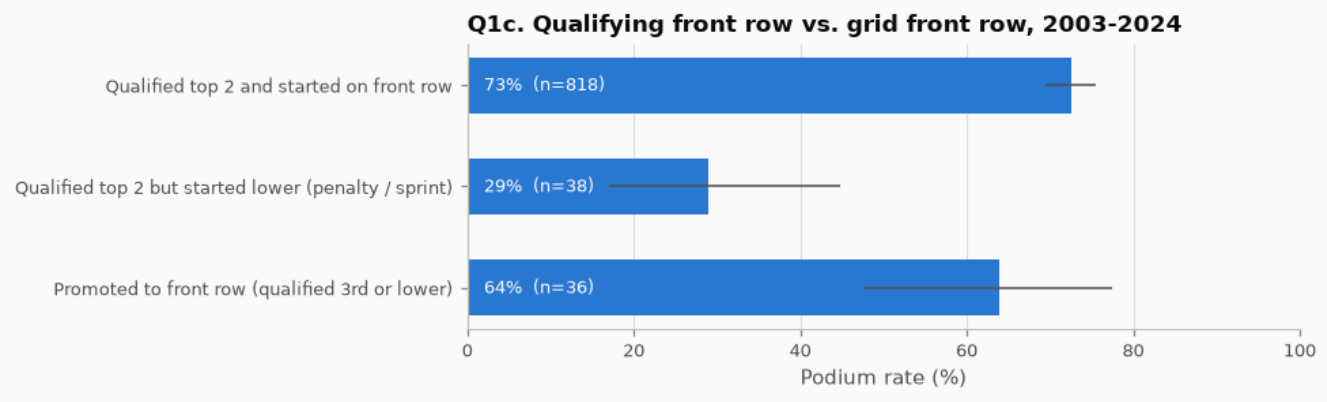
	races_pre	rate_pre	races_hybrid	rate_hybrid	rank_pre	rank_hybrid	change_pts
circuit							
Suzuka Circuit	9	72.2	9	88.9	4	1	16.7
Yas Marina Circuit	5	60.0	11	86.4	10	2	26.4
Circuit de Monaco	11	77.3	10	85.0	1	3	7.7
Circuit de Barcelona-Catalunya	11	68.2	11	81.8	6	4	13.6
Circuit de Spa-Francorchamps	9	72.2	11	81.8	4	4	9.6
Autódromo José Carlos Pace	11	59.1	10	78.9	12	6	19.9
Shanghai International Circuit	10	65.0	7	78.6	8	7	13.6
Circuit Gilles Villeneuve	10	60.0	9	77.8	10	8	17.8
Silverstone Circuit	11	72.7	12	75.0	3	9	2.3
Autodromo Nazionale di Monza	11	77.3	11	72.7	1	10	-4.5
Albert Park Grand Prix Circuit	11	59.1	9	72.2	12	11	13.1
Marina Bay Street Circuit	6	58.3	9	72.2	14	11	13.9
Bahrain International Circuit	9	66.7	12	66.7	7	13	0.0
Hungaroring	11	63.6	11	59.1	9	14	-4.5

```
Overall conversion: 2003–2013 68.0%  ->  2014–2024 76.0%
Spearman rank correlation between the two eras' circuit rankings: rho = 0.30 (p = 0.29, 14 circuits)
```



```
Entries where grid differs from qualifying position, 2003–2024: 27.0%
Front-row qualifiers who did not start on the front row: 38 of 856
Conversion by definition: grid front row 72.2%  vs  qualifying front row 70.7%
```

	starts	podiums	rate	lo	hi
Qualified top 2 and started on front row	818.0	594.0	72.6	69.5	75.6
Qualified top 2 but started lower (penalty / sprint)	38.0	11.0	28.9	17.0	44.8
Promoted to front row (qualified 3rd or lower)	36.0	23.0	63.9	47.6	77.5




```
Rank agreement between grid-based and qualifying-based circuit rankings (2003–2024, >= 10 races): rho = 0.89
```

Answer to Question 1

- **Which circuits convert most often?** Over all seasons a front-row starter reaches the podium in **60%** of starts. Among circuits with at least 10 races, the highest conversion is at Circuit of the Americas (83%), Circuit de Nevers Magny-Cours (81%), Yas Marina Circuit (78%); the lowest at Kyalami (42%), Brands Hatch (39%), Autódromo Internacional Nelson Piquet (35%).
- **Does the ranking change in the hybrid era?** Yes, substantially. Front-row conversion rose overall from **68%** (2003–2013) to **76%** (2014–2024), and the circuit order was reshuffled: the rank correlation between the two eras is only **ρ = 0.30** (p = 0.29), i.e. a circuit's pre-2014 rank says little about its hybrid-era rank. Pre-hybrid leaders: Circuit de Monaco (77%), Autodromo Nazionale di Monza (77%), Silverstone Circuit (73%). Hybrid-era leaders: Suzuka Circuit (89%), Yas Marina Circuit (86%), Circuit de Monaco (85%). Largest gains: Yas Marina Circuit (+26 pts), Autódromo José Carlos Pace (+20 pts), Circuit Gilles Villeneuve (+18 pts). Only 2 of 14 circuits went down: Autodromo Nazionale di Monza (-5 pts), Hungaroring (-5 pts).
- **Qualifying vs. grid.** In 2003–2024 the grid differs from the qualifying position for 27% of entries. Drivers who qualified in the top two but started lower converted only 29% of the time, against 73% for those who kept their front-row slot — so the *grid* is the position that matters, and the two must not be mixed. The circuit ranking itself is similar under both definitions (ρ = 0.89).
- **Caution.** Each circuit–era cell holds only 10–22 front-row starts, so individual circuit rates have wide intervals (see the error bars in Q1a). The overall rise and the lack of rank stability are robust; the exact position of a single circuit is not.

Why the ranking moves. The hybrid era had long spells of one-team dominance (Mercedes 2014–2020, Red Bull 2022–2023) and much better reliability, so the two fastest cars more often simply finished where they started. With so few front-row starts per circuit, a circuit's position in the table mainly reflects *which seasons it hosted* and how dominant the leading car was in them, not a fixed property of the track. This is why circuit identity is **not** used as a raw feature later; a smoothed, past-only circuit conversion rate is tested instead.

Question 2 — Do drivers reach the podium more often at home, after controlling for starting position?

Joins. `results → drivers` gives the driver's nationality; `results → races → circuits` gives the country of the circuit (four tables). The two use different vocabularies (`British` vs. `UK`), so a nationality-to-country mapping table is needed as a fifth input.

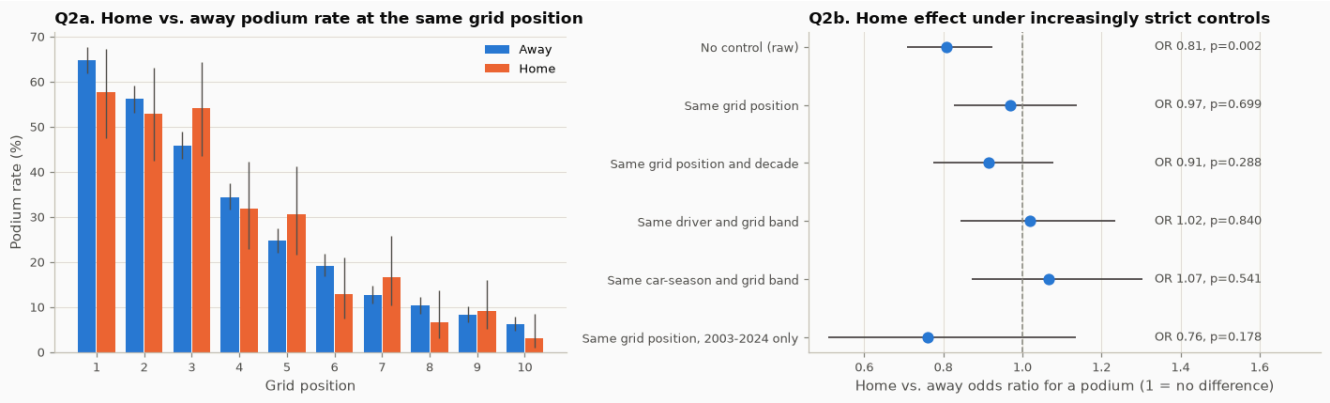
Grain. One row per driver per race, restricted to rows with a real grid slot (`grid > 0`), because "starting position" is undefined otherwise.

Missingness. Nationality and circuit country are never missing. Nationalities with no circuit in the data (for example Finnish or Danish) are always "away". Dual nationalities count as home in either country.

Control. Home drivers could do better simply because they tend to be in better cars and qualify higher. We therefore compare home and away drivers **at the same grid position**, in three ways of increasing strictness.

	odds_ratio	lo	hi	p_value	strata
No control (raw)	0.809	0.708	0.924	0.002	1.0
Same grid position	0.969	0.826	1.137	0.699	20.0
Same grid position and decade	0.914	0.775	1.078	0.288	159.0
Same driver and grid band	1.020	0.843	1.235	0.840	727.0
Same car-season and grid band	1.066	0.871	1.305	0.541	1153.0
Same grid position, 2003-2024 only	0.761	0.510	1.135	0.178	20.0

```
Logistic regression, home effect given grid and decade: odds ratio 0.94 (95% CI 0.80–1.11, p = 0.452)
```



Answer to Question 2 — No. Once starting position is held fixed, there is no home advantage.

- **Raw comparison.** Drivers reach the podium in **11.4%** of home races against **13.8%** elsewhere (odds ratio 0.81, 95% CI 0.71–0.92). Taken at face value, drivers do *worse* at home.
- **Why the raw number misleads.** Home drivers start further back on average (mean grid 13.4 vs 11.7). The raw gap is a starting-position gap, not a home effect.
- **Same grid position.** Comparing only drivers who started from the same grid slot, the odds ratio is **0.97** (95% CI 0.83–1.14): no detectable difference (the interval includes 1).
- **Same driver, similar grid position.** Comparing each driver only with himself, home vs. away: **1.02** (95% CI 0.84–1.23): no detectable difference (the interval includes 1).
- **Same car and season, similar grid position. 1.07** (95% CI 0.87–1.30): no detectable difference (the interval includes 1).
- **Modern era only (2003–2024), same grid position. 0.76** (95% CI 0.51–1.13): no detectable difference (the interval includes 1).

- A logistic regression with grid position and decade as controls gives the same picture: odds ratio 0.94 (95% CI 0.80–1.11).

Interpretation. The raw comparison points the "wrong" way because of *who* races at home. Across F1 history, home races attracted many local one-off and privateer entries — drivers who entered only their national Grand Prix, in uncompetitive cars, and started at the back (the Indianapolis 500 of the 1950s, counted in the championship, is the extreme case). These entries pull the home podium rate down. This is a selection effect in the data, not evidence that racing at home hurts.

Holding grid position fixed removes that selection, and every controlled estimate sits close to 1 with an interval that includes 1. Panel Q2a shows the same thing: at any given grid slot the home and away bars are close and their intervals overlap. The data give no support for a home-race effect on podium chances in either direction.

For modeling, this predicts that a home-race flag adds little once grid position is known. It is kept as a candidate feature and tested in Section 7 rather than dropped on assumption.

Question 3 — Which constructors have the highest mechanical-retirement rate in 2014–2021 vs. 2022–2024?

Joins. `results → status` gives the reason each race ended; `results → constructors` gives the team; `results → races` gives the season (four tables).

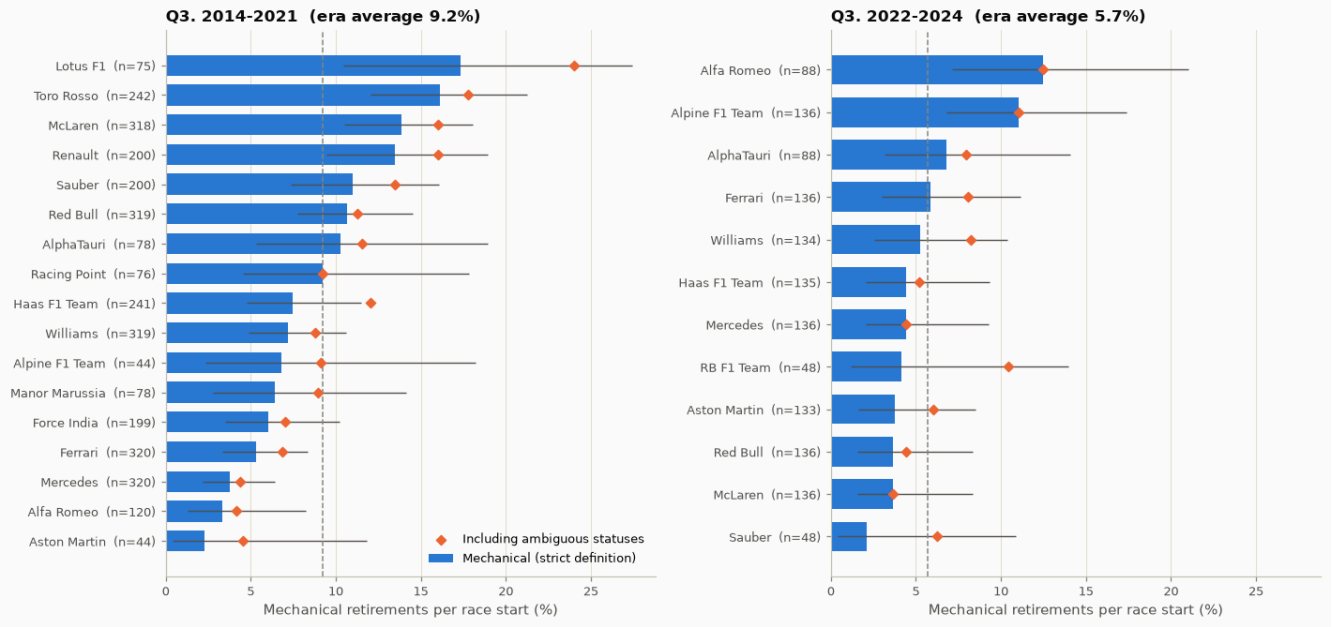
Definition. The `status` table has 139 free-text reasons, so "mechanical retirement" has to be built from them. Every status is assigned to exactly one class:

Class	Meaning	Counts as mechanical retirement?
finished	`Finished`, `+n Laps`, `Not classified`	No
mechanical	A car component failed (engine, power unit, gearbox, brakes, hydraulics, …)	**Yes**
crash	Accident, collision, spin, damage, debris, puncture	No
driver	Illness, injury	No
regulation	Disqualified, excluded, underweight, safety decision	No
not started	Withdrew, did not qualify, 107% rule	No — and removed from the denominator
ambiguous	Could be a failure, damage or a team/pit error (`Retired`, wing failures, wheel / wheel nut, tyre, out of fuel, …)	No in the main analysis; **yes** in the sensitivity check

Rate. Mechanical retirements ÷ race starts. A race start is an entry that was not withdrawn or excluded before the race.

Grain. One row per driver per race, aggregated to constructor × era. **Missingness.** `statusId` is never missing and every value matches the `status` table (Section 2.4). **Minimum sample.** A constructor needs at least 40 starts in an era (one full season of two cars) to be ranked.

```
Overall mechanical-retirement rate per start: {'2014–2021': '9.2%', '2022–2024': '5.7%'}
```

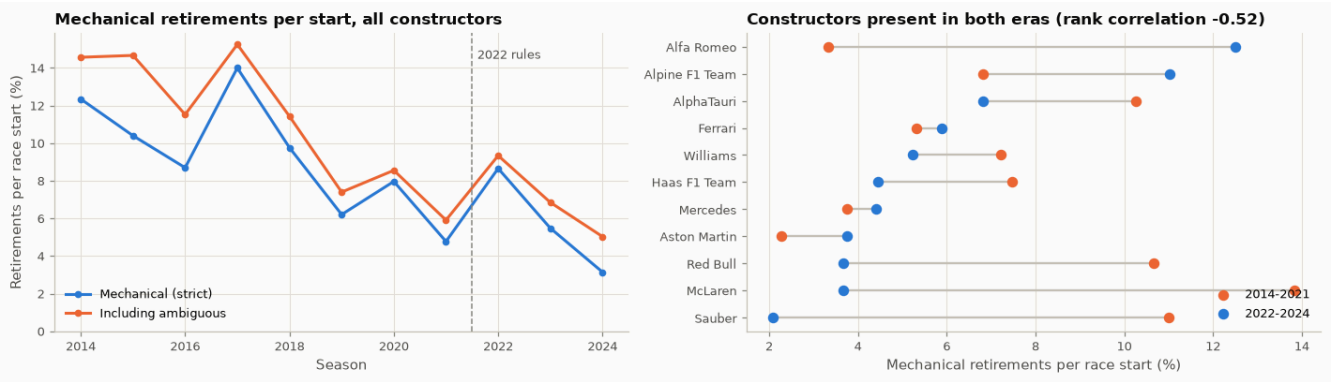


2014–2021: highest mechanical-retirement rates

	starts	mech	mech_broad	rate	rate_broad	lo	hi
constructor							
Lotus F1	75	13	18	17.3	24.0	10.4	27.4
Toro Rosso	242	39	43	16.1	17.8	12.0	21.3
McLaren	318	44	51	13.8	16.0	10.5	18.1
Renault	200	27	32	13.5	16.0	9.4	18.9
Sauber	200	22	27	11.0	13.5	7.4	16.1
Red Bull	319	34	36	10.7	11.3	7.7	14.5

2022–2024: highest mechanical-retirement rates

	starts	mech	mech_broad	rate	rate_broad	lo	hi
constructor							
Alfa Romeo	88	11	11	12.5	12.5	7.1	21.0
Alpine F1 Team	136	15	15	11.0	11.0	6.8	17.4
AlphaTauri	88	6	7	6.8	8.0	3.2	14.1
Ferrari	136	8	11	5.9	8.1	3.0	11.2
Williams	134	7	11	5.2	8.2	2.6	10.4
Haas F1 Team	135	6	7	4.4	5.2	2.1	9.4



```
Rank agreement strict vs. broad definition: {'2014–2021': np.float64(0.97), '2022–2024':
np.float64(0.7)}
```

Answer to Question 3

- **2014–2021.** Highest mechanical-retirement rates per race start: **Lotus F1** 17.3% (13/75), **Toro Rosso** 16.1% (39/242), **McLaren** 13.8% (44/318). Lowest: Aston Martin 2.3%, Alfa Romeo 3.3%. Era average: 9.2%.
- **2022–2024.** Highest: **Alfa Romeo** 12.5% (11/88), **Alpine F1 Team** 11.0% (15/136), **AlphaTauri** 6.8% (6/88). Lowest: Sauber 2.1%, McLaren 3.7%. Era average: 5.7%.
- **Change between eras.** Reliability improved across the field: the average rate fell from 9.2% to 5.7% of starts. For the constructors present in both eras, the rank correlation is -0.52 (p = 0.10), which measures how far the reliability order of one era carries over to the next (1 = same order, 0 = unrelated).
- **Sensitivity.** Counting the ambiguous statuses as mechanical raises the rates but leaves the 2014–2021 order almost unchanged (rank agreement 0.97). In 2022–2024 agreement is lower (0.70) because counts are small — a handful of borderline retirements can move a team several places — so the top of that list is reliable but the middle is not.

Reading the result. The 2014–2021 list is led by teams that ran the Renault and Honda power units of the early hybrid years (Lotus, Toro Rosso, McLaren, Renault, Red Bull), which is consistent with the well-documented reliability problems of those units; this is context, not something the data alone proves. In 2022–2024 rates are lower overall and the intervals are wide (a team has only about 130 starts in three seasons), so differences between mid-table teams in that era are within noise.

Two caveats. First, team identity follows `constructorId`: a renamed team (Toro Rosso → AlphaTauri → RB, Force India → Racing Point → Aston Martin, Renault → Alpine, Sauber → Alfa Romeo) appears as separate constructors. Second, the result depends on how free-text statuses are classed; that is why the classification is explicit in code and a sensitivity check is shown.

The same status classification is reused in Section 6 to build a past-only constructor reliability feature.

---

6. Leakage audit and feature engineering

6.1 Which fields are known at the prediction cutoff?

The prediction is made **after qualifying and before the race starts**. A field may be used as a feature only if its value is fixed by that moment. The audit below classifies every source of information. The test applied to each field is: *"Could this value still change, or does it only come into existence, once the race is running?"*

table	fields	available at cutoff?	reason	decision
results	grid	pre-race	Final starting grid is published before the start	Used
results	constructorId, number	pre-race	Entry list is known before the weekend	Used for joins / team features
results	position, positionText, positionOrder	POST-RACE	This is the race outcome; positionOrder defines the target	Target only; past races feed form features
results	points	POST-RACE	Awarded for the finishing position	Excluded
results	laps, time, milliseconds	POST-RACE	Race distance and race time	Excluded
results	fastestLap, rank, fastestLapTime, fastestLapSpeed	POST-RACE	Set during the race	Excluded
results	statusId	POST-RACE	Finish / retirement reason	Excluded; past races feed reliability features
qualifying	position, q1, q2, q3	pre-race	Qualifying ends before the cutoff	Used
driver_standings	points, position, wins (same raceId)	POST-RACE	Standings rows are published AFTER the race they are keyed to	Excluded; the PREVIOUS round's row is used
constructor_standings	points, position, wins (same raceId)	POST-RACE	Same as driver standings	Excluded; the PREVIOUS round's row is used
constructor_results	points	POST-RACE	Team points scored in the race	Excluded
lap_times	all columns	POST-RACE	Recorded lap by lap during the race	Excluded
pit_stops	all columns	POST-RACE	Happen during the race	Excluded
sprint_results	all columns	ambiguous	Sprint is run before the race, but its place relative to qualifying changed between 2021 and 2024	Excluded to keep one consistent cutoff
races	year, round, circuitId, date	pre-race	Calendar is fixed in advance	Used
drivers	dob, nationality	pre-race	Static driver attributes	Used
circuits	country	pre-race	Static circuit attribute	Used

The two traps that are easy to miss.

1. **Standings are keyed to the race they follow.** `driver_standings` for `raceId = X` already includes the points scored *in* race X. Joining standings on the same `raceId` would hand the model the answer. We therefore join the standings of the **previous round**.
2. **Form features must use past races only.** A "podium rate over the last five races" that includes the current race contains the target. Every rolling or cumulative statistic below is shifted by one race before it is used.

Section 6.3 tests both mechanically.

6.2 Feature construction

All features are built by one function, `build_features`, so the exact same code can be re-run on altered data for the leakage test. Features are computed on the **full 1950–2024 history** (so that a driver in 2003 already has a record) and the modeling table then keeps 2003–2024.

Group	Feature	Definition	Why it is valid at the cutoff
Grid	`grid_pos`	Starting slot; `grid = 0` recoded to "behind the last car"	Grid is published before the start
	`pit_lane_start`	1 if `grid = 0`	Same
	`grid_drop`	`grid_pos − quali_pos` (positive = lost places to a penalty)	Both known before the start
Qualifying	`quali_pos`	Qualifying classification	Qualifying is over
	`quali_gap_pct`	Best qualifying lap vs. the fastest lap of the session, in %	Same
	`mate_quali_gap_pct`	Best lap vs. the team-mate's best lap, in % (driver vs. same car)	Same
	`team_best_gap_pct`	Smaller of the two cars' gaps to pole (car pace this weekend)	Same
	`quali_missing`	1 if the driver has no qualifying record	Same
Driver form	`drv_podium_rate_5`, `drv_avg_finish_5`	Podium rate and mean finishing position over the driver's **previous** 5 races	Past races only (shifted)
	`drv_dnf_rate_10`	Share of the previous 10 races not finished	Past races only
	`drv_career_podium_rate`, `drv_career_starts`	Career podium rate and number of races **before** this one	Past races only
	`drv_circuit_podium_rate`, `drv_circuit_starts`	Same, restricted to earlier visits to this circuit	Past races only
Constructor form	`con_podium_rate_5`, `con_avg_finish_5`	Team podium rate and mean finish over the team's previous 5 races (both cars)	Past races only
	`con_mech_rate_10`	Team mechanical-retirement rate over the previous 10 races (classification from Question 3)	Past races only
Standings	`drv_stand_pos`, `drv_points_share`, `drv_wins`	Championship position, points as a share of the leader's points, and wins **after the previous round**	Previous round's standings
	`con_stand_pos`, `con_points_share`	Same for the constructor	Previous round's standings
Context	`is_home`	Circuit is in the driver's home country (Question 2)	Static
	`season_progress`	Round ÷ rounds in the season	Calendar
	`field_size`	Number of entries in the race	Entry list
	`driver_age`	Age on race day	Static
	`circuit_front_conv_prior`	Smoothed front-row→podium rate at this circuit in **earlier** races (Question 1)	Past races only

Design choices.

- **Points are expressed as a share of the leader's points**, not raw points. The points system changed in 2010 (a win went from 10 to 25 points), so raw points are not comparable across seasons, while a share is.
- **At round 1 there are no standings yet**, so the final standings of the previous season are carried over. A driver or team with no standing at all (a rookie, a new team) is placed one position behind the last ranked entry, with zero points share. This is a domain rule, not a statistical fill: "unranked" genuinely means "behind everyone ranked".

- **Qualifying gaps are capped** at 10% (gap to pole) and ±5% (gap to team-mate). Larger values come from sessions where the weather changed between runs, and say nothing about pace.
- **Rookies have no form history.** Their form features are left missing here and filled inside the model pipeline (Section 9), where the fill value is learned from training data only.

```
Feature table built in 1.5s: 26,668 rows x 28 features
```

6.3 Leakage tests

Reading the code is not proof, so leakage safety is tested by experiment.

Test 1 — "scramble the future". Pick a race. In a copy of the data, destroy every post-race fact for that race **and every later race**: shuffle finishing positions, overwrite the status, and overwrite the standings published after those races. Rebuild all features. If the features of the chosen race use only pre-race information, they must be **exactly unchanged**. This is repeated for several races.

Test 2 — negative control. The same test is applied to a deliberately leaky feature (standings joined on the *same* `raceId`). It must fail; otherwise the test would not be able to detect leakage at all.

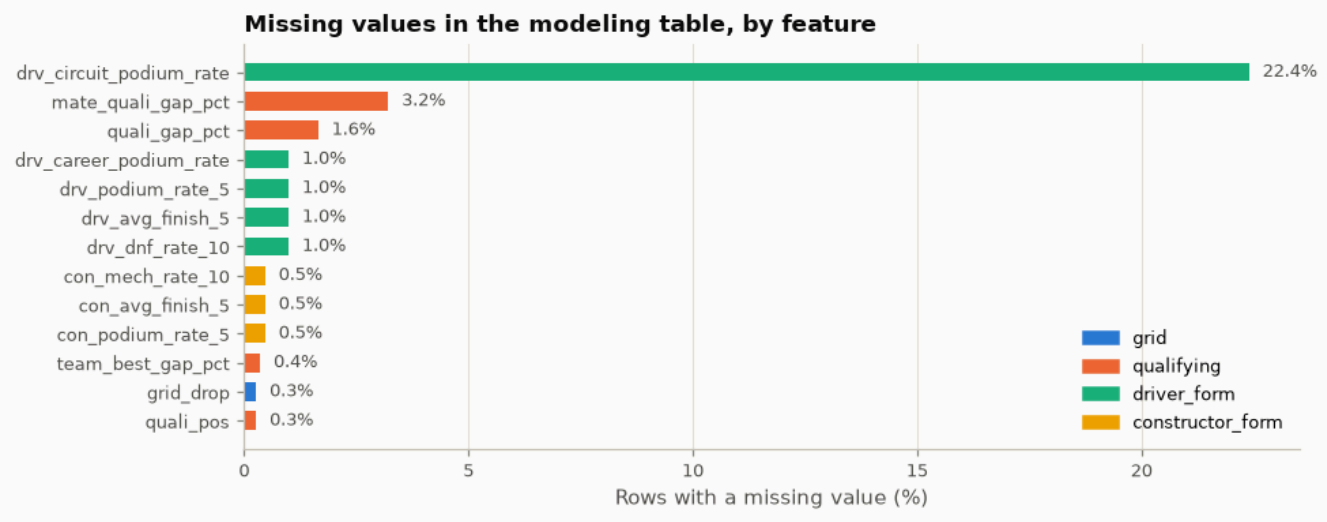
	race	rows	features changed	leak-free	negative control changed
0	2017 R20	20	0	True	True
1	2020 R8	20	0	True	True
2	2013 R4	22	0	True	True
3	2022 R8	20	0	True	True
4	2013 R6	22	0	True	True
5	2007 R3	22	0	True	True

```
PASS: all 28 features are unchanged when current and future outcomes are scrambled;
      the deliberately leaky feature is detected in every case.
```

Result. For every sampled race, all features are identical before and after the future is scrambled, and the deliberately leaky join is caught each time. The feature set is leakage-safe by test, not only by intention.

6.4 The modeling table

```
Modeling table: 8,970 rows = 428 races x drivers, seasons 2003–2024
Unique (raceId, driverId): True   |   podium share: 14.3%
```



Missingness in the modeling table. What remains missing has a clear meaning:

- `drv_circuit_podium_rate` — the driver has never raced at this circuit before (first visit). This is the largest group.
- `mate_quali_gap_pct` — the team-mate set no qualifying time, or the team ran one car.
- Form features — a driver's or team's first race (no history yet).
- Qualifying features — the few entries with no qualifying record.

These are filled inside the model pipeline with values learned from the training set (Section 9). `drv_circuit_starts`, `drv_career_starts` and `quali_missing` already tell the model *why* a value is missing.

---

7. Train / validation / test split

Choice: a chronological split by season.

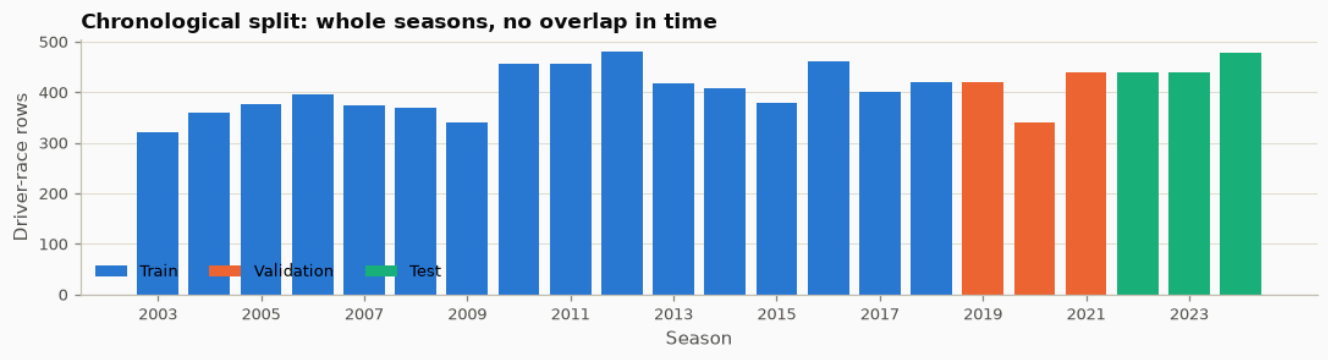
Set	Seasons	Purpose
Train	2003–2018	Fit model parameters
Validation	2019–2021	Choose features, hyperparameters and the decision threshold
Test	2022–2024	Final, untouched estimate of performance

Why chronological, and not a random split?

1. **It matches how the model would be used.** The model is trained on past seasons and applied to races that have not happened yet. A random split would train on 2024 races and "predict" 2015, which never happens in practice.
2. **Rows are not independent.** The 20 rows of one race share the same weather, incidents and safety cars, and exactly three of them are positive. A random row split would put some drivers of a race in training and the rest in test, letting the model learn about a race it is being tested on. Splitting by season keeps every race, and every season, whole.
3. **The form features carry the past forward.** With a random split, a test row's "last five races" would overlap with training rows' targets from the same weeks.
4. **The test set is a hard, honest one.** 2022 introduced new technical regulations, so the test seasons differ from the training seasons. This measures how the model copes with a change of era (a limitation discussed in Section 15), instead of hiding it.

The split boundaries are at season ends so that no season is divided. A numerical comparison with a random split is shown in Section 10.8.

	seasons	races	rows	podiums	podium_share	share_of_rows
split						
train	2003-2018	300	6411	900	14.0	71.5
val	2019-2021	60	1200	180	15.0	13.4
test	2022-2024	68	1359	204	15.0	15.2



The three sets are disjoint in races and strictly ordered in time, and the podium share is nearly the same in each (it is fixed by the field size), so the sets are comparable.

---

8. Feature selection: evidence for each choice

All evidence in this section is computed on the **training seasons only** (and, for trials, scored on validation). The test seasons are not looked at.

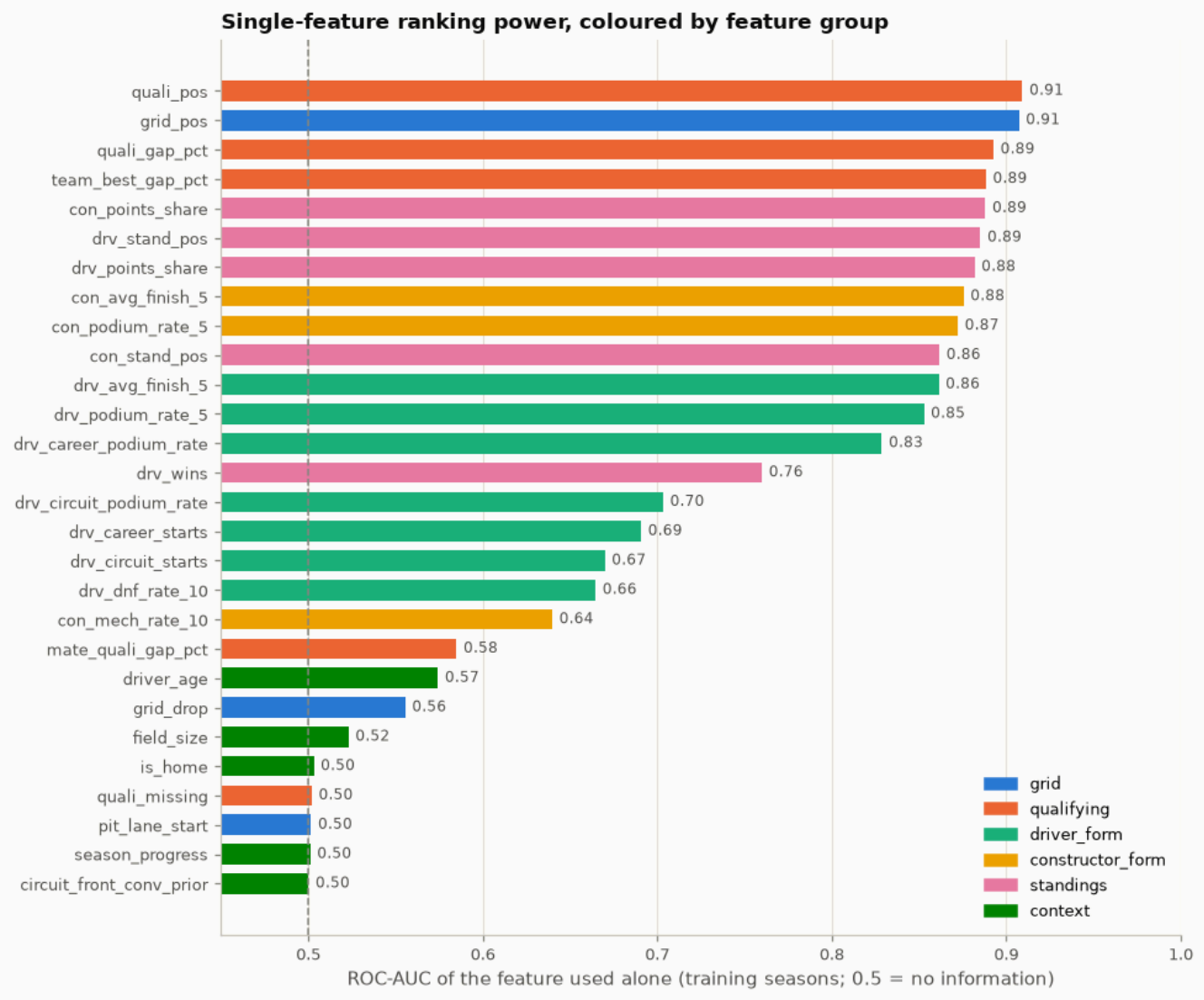
Two kinds of evidence are used, as the milestone allows:

- **Hypothesis-driven** — a plot or statistic from EDA that shows the feature is related to the target (8.1–8.3).
- **Trial-and-error** — the feature set is changed, the model is re-trained, and the validation score is compared (8.4).

8.1 How strongly is each feature related to the target on its own?

For every feature we report, on the training set:

- **Spearman correlation** with the target (direction and strength of a monotonic relationship),
- **univariate ROC-AUC** — how well the feature alone ranks podium rows above non-podium rows (0.5 = useless, 1 = perfect; direction ignored),
- **mutual information** — picks up relationships that are not monotonic.

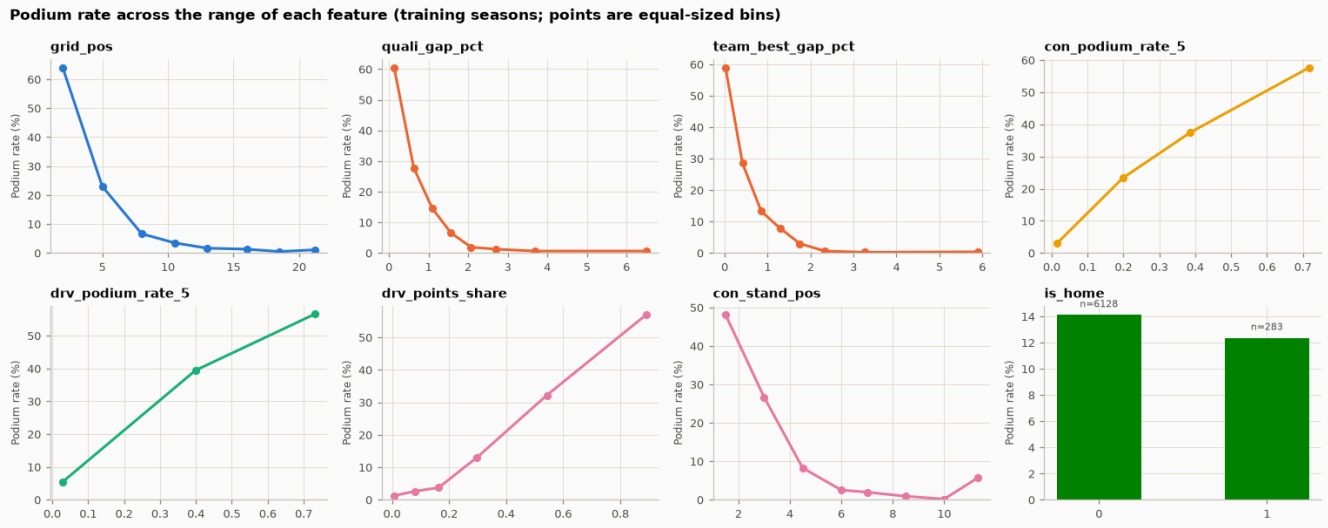


Reading the chart.

- **Grid and qualifying** features are the strongest on their own, as expected from Section 4.5: starting position alone separates podium from non-podium rows very well.
- **Constructor form and standings** come next. The car matters: how the team has been performing recently is almost as informative as where this driver qualified.
- **Driver form** is informative but weaker than team form, which fits the common observation that results in F1 depend heavily on the car.
- **Context features** are weak alone, and for most of them that is unavoidable. `field_size`, `season_progress` and `circuit_front_conv_prior` are **race-level**: every driver in a race has the same value, so on their own they cannot rank one driver above another and must score 0.5. They can only help by changing the *meaning* of other features (standings at round 2 are less reliable than at round 20; a front-row start is worth more at some circuits). `is_home` is close to 0.5, in line with Question 2 (no home effect).
- **Flags** (`pit_lane_start`, `quali_missing`) also sit at 0.5 because they are switched on for fewer than 1% of rows; they exist to describe those rare rows correctly, not to rank the field.
- Whether the weak-alone features earn their place in combination is tested in 8.4 and in the ablation (Section 12).

8.2 The shape of the relationship

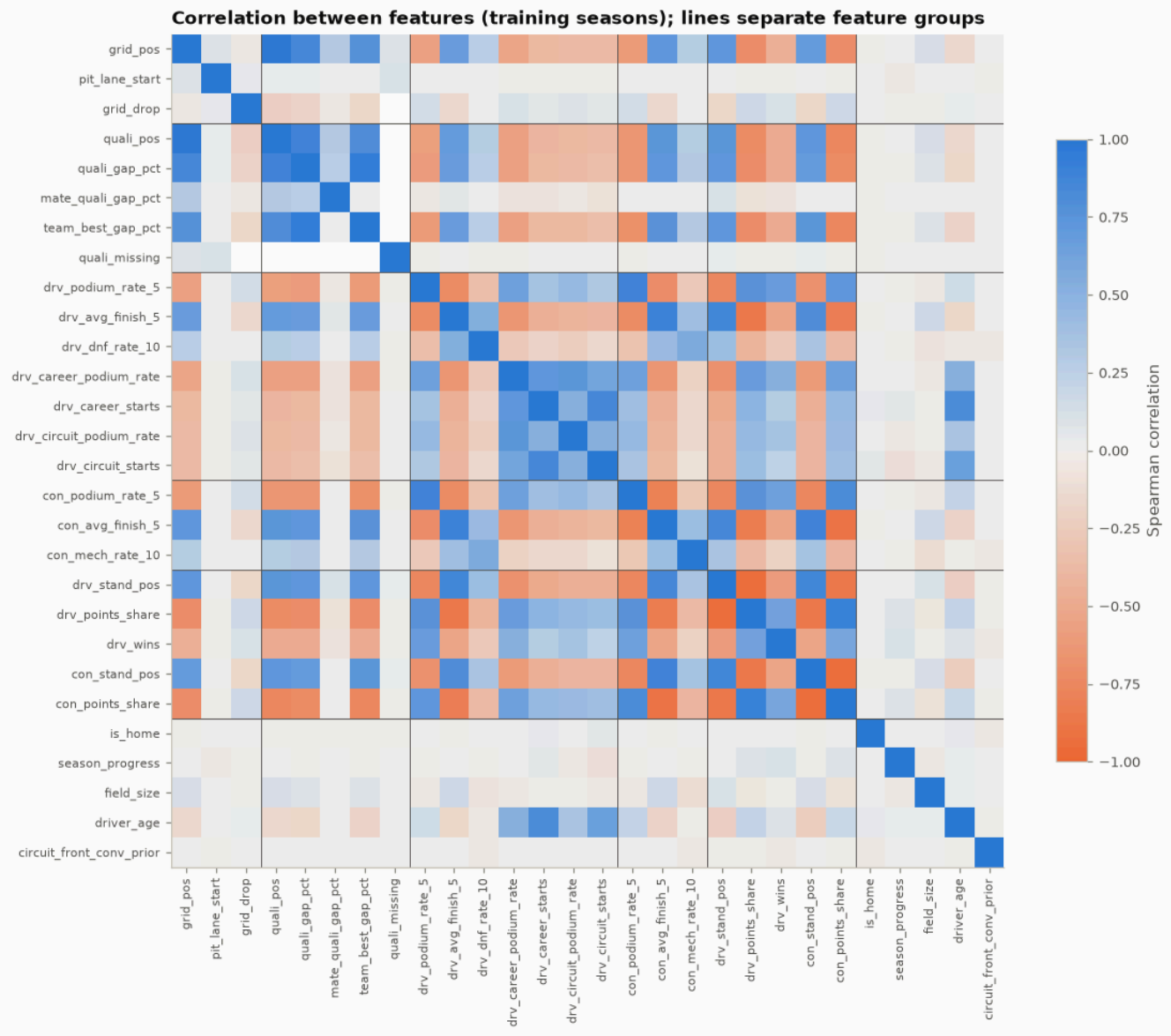
A correlation gives one number; the plots below show the podium rate across the range of eight features, to check the direction and whether the relationship is linear.



What the shapes say.

- Every relationship runs in the expected direction (closer to the front, smaller gap to pole, better recent results → higher podium rate).
- Most are **strongly non-linear**: the podium rate is near zero over most of the range and then rises steeply at the front. This is a reason to try tree-based models and a neural network, which can represent such shapes, in addition to a linear model.
- `is_home` shows almost no difference between the two bars, again matching Question 2.

8.3 Redundancy between features



	feature_a	feature_b	spearman
0	grid_pos	quali_pos	0.97
1	drv_stand_pos	drv_points_share	-0.95
2	quali_gap_pct	team_best_gap_pct	0.93
3	con_stand_pos	con_points_share	-0.92
4	drv_points_share	con_points_share	0.91
5	drv_avg_finish_5	con_avg_finish_5	0.90
6	con_avg_finish_5	con_points_share	-0.89
7	quali_pos	quali_gap_pct	0.88
8	con_avg_finish_5	con_stand_pos	0.88
9	drv_podium_rate_5	con_podium_rate_5	0.87
10	drv_stand_pos	con_points_share	-0.87
11	drv_stand_pos	con_stand_pos	0.86
12	drv_avg_finish_5	drv_stand_pos	0.86
13	grid_pos	quali_gap_pct	0.85
14	drv_avg_finish_5	drv_points_share	-0.85
15	drv_points_share	con_stand_pos	-0.84
16	con_avg_finish_5	drv_stand_pos	0.84
17	con_avg_finish_5	drv_points_share	-0.83
18	drv_career_starts	drv_circuit_starts	0.83
19	drv_avg_finish_5	con_points_share	-0.81
20	drv_career_starts	driver_age	0.81
21	con_podium_rate_5	con_points_share	0.81
22	con_podium_rate_5	con_avg_finish_5	-0.80

What this shows. Several features are strongly correlated, in two families: (1) `grid_pos`, `quali_pos` and the qualifying gaps, and (2) the "how good is this car/driver lately" features — standings and rolling form. This is expected: they are different measurements of the same underlying things (pace this weekend; strength this season).

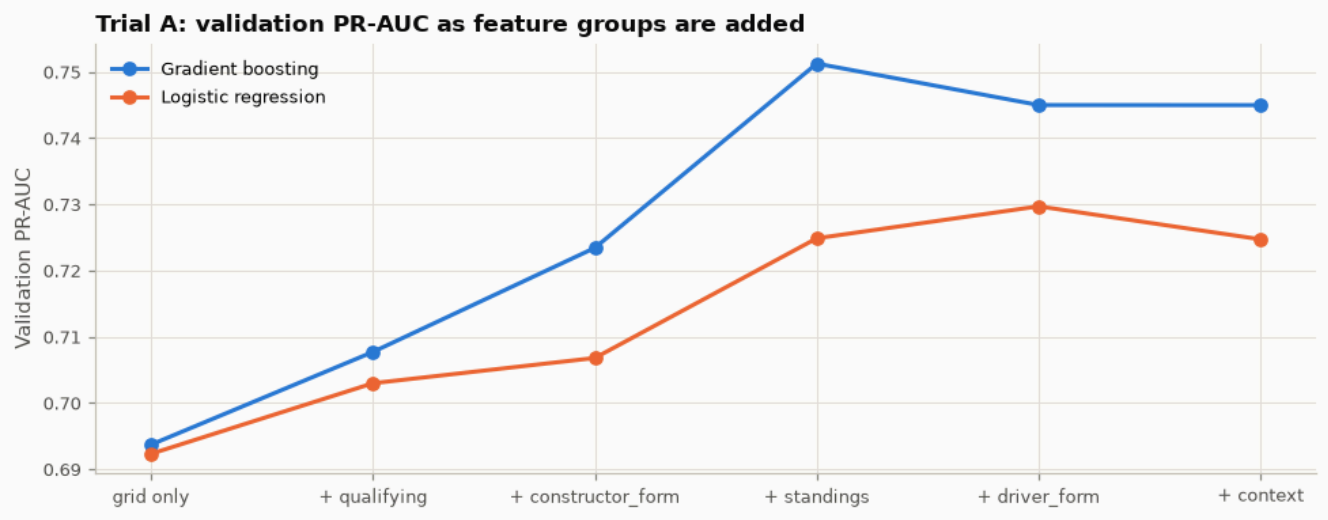
Consequences.

- Correlated features do not harm the predictions of regularised or tree models, but they **make coefficients and importances unstable**: importance is shared between near-duplicates. For this reason the ablation in Section 12 removes whole **groups**, not single features, and the explanations in Section 13 are read at group level.
- `grid_pos` and `quali_pos` are kept together on purpose. They differ in about a quarter of entries, and Question 1 showed that the difference matters: a driver who qualified on the front row but was moved back converts far less often. `grid_drop` makes that difference explicit.

8.4 Trials: does each group, and each encoding choice, earn its place?

Trial A — add the groups one at a time. Starting from the grid alone, each group is added and the model re-trained. This shows the value each group adds *on top of* what is already there. Two model types are used so that a conclusion does not depend on one model.

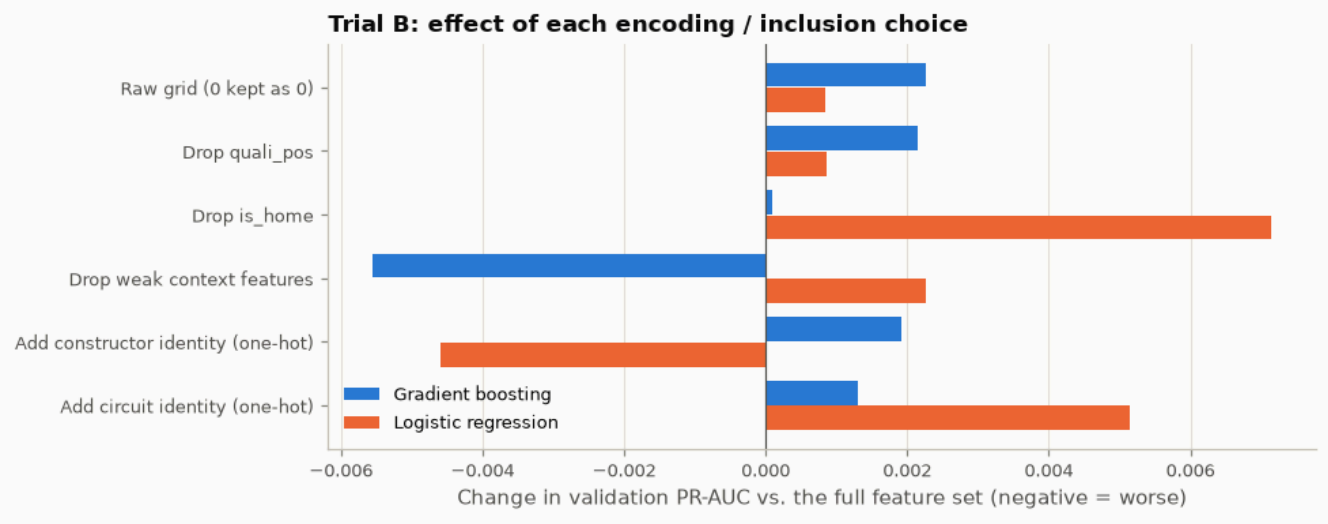
		PR-AUC		ROC-AUC	
	model	Gradient boosting	Logistic regression	Gradient boosting	Logistic regression
features	n_features				
grid only	3	0.6937	0.6923	0.9234	0.9232
+ qualifying	8	0.7077	0.7030	0.9255	0.9198
+ constructor_form	11	0.7234	0.7068	0.9371	0.9260
+ standings	16	0.7513	0.7249	0.9407	0.9307
+ driver_form	23	0.7450	0.7297	0.9391	0.9308
+ context	28	0.7450	0.7247	0.9389	0.9309



Trial B — encoding and inclusion choices. Each row changes one thing relative to the full feature set and reports the change in validation PR-AUC.

Variant	Question it answers
raw `grid` (0 kept as 0) instead of `grid_pos` + `pit_lane_start`	Does recoding `grid = 0` matter?
drop `quali_pos`	Is it redundant given `grid_pos`?
drop `is_home`	Question 2 suggests it is uninformative
drop weak context (`field_size`, `season_progress`, `driver_age`)	Do the weak-alone context features help in combination?
add constructor identity (one-hot)	Should the team *name* be a feature, instead of team form?
add circuit identity (one-hot)	Should the circuit *name* be a feature?

model	Gradient boosting	Logistic regression	n_features
variant			
Full feature set (reference)	0.7450	0.7247	28
Raw grid (0 kept as 0)	0.7472	0.7255	27
Drop quali_pos	0.7471	0.7256	27
Drop is_home	0.7451	0.7319	27
Drop weak context features	0.7394	0.7270	25
Add constructor identity (one-hot)	0.7469	0.7201	57
Add circuit identity (one-hot)	0.7463	0.7298	59



How to read these trials. The validation set has 60 races and 180 podiums, so differences of a few thousandths in PR-AUC are within noise. Only changes that are clearly larger than that, or that agree across both model types, are treated as evidence.

Decisions.

- **Grid and qualifying carry most of the signal; team form and standings add a clear further gain** (Trial A). Later groups add less because they overlap with what is already included — which is what the correlation matrix predicted. Small *decreases* when a group is added on top are within noise for this sample size.
- **Team and circuit identity are not used.** One-hot identity columns add 30–40 columns for no reliable gain. More importantly, a team's *name* is not a stable signal across seasons — the same constructor can be dominant in one period and mid-field in another, and teams are renamed — whereas rolling form and standings describe how strong the team is *now*. They also cannot handle a team or circuit that is new in the test seasons.
- **`grid = 0` is recoded.** The recoded version is kept on the grounds of meaning (Section 4.5: these rows have a 0% podium rate, yet 0 sorts ahead of pole). The trial shows what difference this makes in practice; with under 1% of rows affected the measured effect is necessarily small.
- **`quali_pos`, `is_home` and the weak context features are kept.** Removing them does not give a consistent improvement across both models, so there is no evidence-based reason to drop them, and a single fixed feature set for all models is a milestone requirement. Their actual contribution is measured again, on the final tuned models, in the ablation (Section 12).

Selected feature set: the 28 features in the six groups of Section 6.2.

---

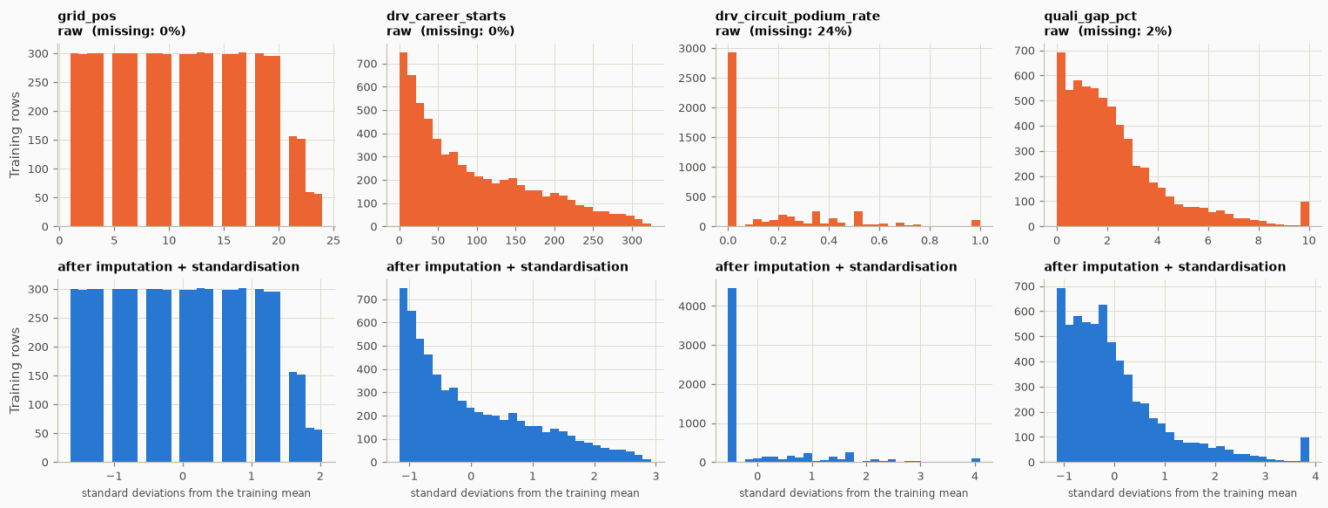
9. Pre-processing and order experiments

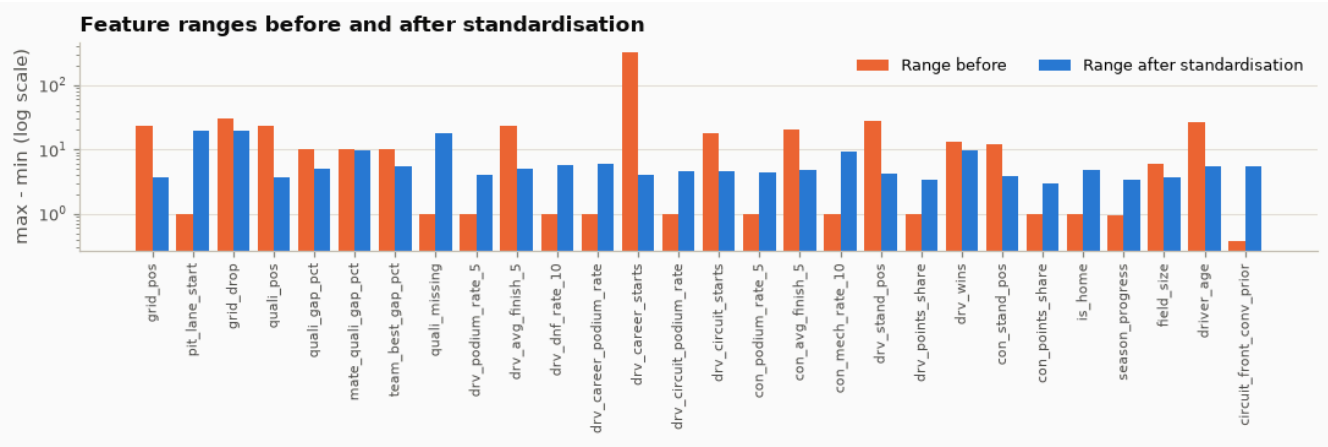
9.1 The pre-processing pipeline and its effect on the data

Two steps are applied, in this order, **inside a pipeline that is fitted on the training seasons only**:

1. **Median imputation.** Missing values are replaced by the feature's training-set median. The median is used instead of the mean because several features are heavily skewed (many drivers have a podium rate of exactly 0), so the mean would not be a typical value.
2. **Standardisation.** Each feature is rescaled to mean 0 and standard deviation 1 using training-set statistics. The features live on very different scales (`drv_career_starts` goes up to 400, rates lie between 0 and 1). Logistic regression with regularisation and the neural network both need comparable

scales: otherwise the penalty and the gradient steps are dominated by the features with the largest numbers. Tree models are unaffected by scaling, but the same pipeline is used for all models so that they see identical inputs.





Effect of the steps.

- **Missing values:** all removed. The imputed rows appear as a single tall bar at the median (most visible for `drv_circuit_podium_rate`, where about a fifth of the rows are first visits to a circuit).
- **Range and scale:** before, feature ranges span three orders of magnitude (from 1 to about 400); after, all features are on a comparable scale of a few standard deviations, with mean 0 and standard deviation 1.
- **Shape:** standardisation is a shift and a rescale, so it does **not** change the shape of a distribution. Skewed features stay skewed. This is one more reason to include non-linear models.

Alternatives tried. The table below compares imputation and scaling variants on validation.

	PR-AUC		ROC-AUC	
model	**Gradient boosting**	**Logistic regression**	**Gradient boosting**	**Logistic regression**
pre-processing				
Median impute + standardise (chosen)	0.7450	0.7247	0.9389	0.9309
Mean impute + standardise	0.7426	0.7257	0.9387	0.9313
Median impute + missing flags + standardise	0.7458	0.7289	0.9390	0.9365
Median impute + robust scaling	0.7450	0.7259	0.9389	0.9306
Median impute, no scaling	0.7450	0.7267	0.9389	0.9307

Reading the table. The variants differ very little: gradient boosting is unaffected by the choice of scaler (as expected for trees), and logistic regression changes only in the third decimal place. Automatic missing-value flags do not add a reliable gain, because the feature set already contains explicit "why is it missing" features (`quali_missing`, `drv_career_starts`, `drv_circuit_starts`). The simple, standard choice — median imputation followed by standardisation — is therefore kept; nothing more complicated is justified by the evidence.

9.2 Pre-processing-order experiments

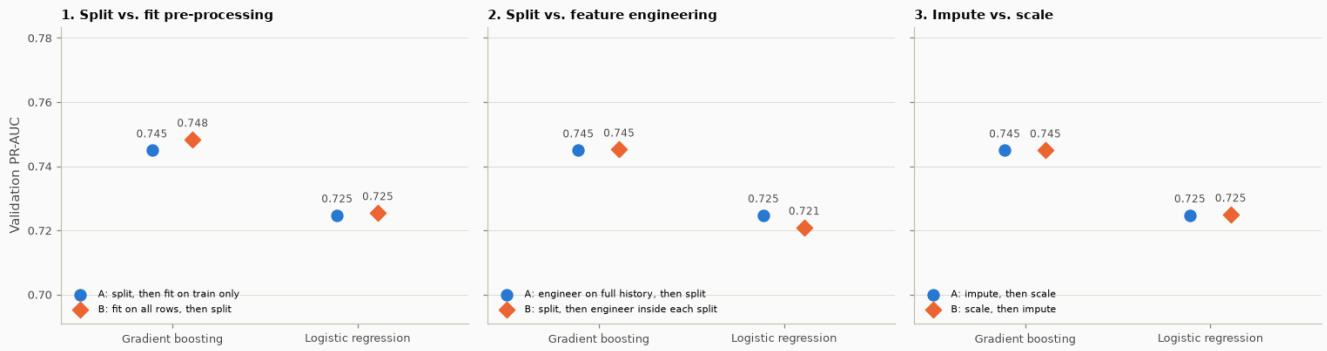
The *order* of pipeline steps can matter as much as the steps themselves. Three order questions are tested. Each compares two orders on the same validation seasons.

#	Order question	Order A (used)	Order B (alternative)
1	Split vs. fitting the pre-processing	Split first, then fit imputer and scaler on **train only**	Fit imputer and scaler on **all rows**, then split
2	Split vs. feature engineering	Build form features on the **full history**, then split	Split first, then build form features **inside each split**
3	Imputation vs. scaling	**Impute → scale**	**Scale → impute**

```
Experiment 1: fitting on all rows moves the scaler means by at most 0.264 training standard deviations (field_size).
```

```
Experiment 3: largest difference between the two orders in any transformed value: 0.501 (drv_circuit_podium_rate).
```

		PR-AUC		ROC-AUC	
	model	**Gradient boosting**	**Logistic regression**	**Gradient boosting**	**Logistic regression**
experiment	**order**				
1. Split vs. fit pre-processing	**A: split, then fit on train only**	0.7450	0.7247	0.9389	0.9309
	B: fit on all rows, then split	0.7483	0.7255	0.9381	0.9308
2. Split vs. feature engineering	**A: engineer on full history, then split**	0.7450	0.7247	0.9389	0.9309
	B: split, then engineer inside each split	0.7452	0.7209	0.9368	0.9293
3. Impute vs. scale	**A: impute, then scale**	0.7450	0.7247	0.9389	0.9309
	B: scale, then impute	0.7450	0.7249	0.9389	0.9308



Results of the order experiments (change in validation PR-AUC when moving from order A to order B)

Experiment	Logistic regression	Gradient boosting
1. Fit pre-processing on all rows instead of train only	+0.0007	+0.0033
2. Engineer features inside each split instead of on full history	-0.0038	+0.0002
3. Scale then impute instead of impute then scale	+0.0002	+0.0000

Interpretation. With 60 validation races, differences below about 0.01 PR-AUC cannot be told apart from noise. None of the three order changes produces a difference clearly larger than that, in either direction. The orders are therefore chosen on correctness, and the experiments show what each choice costs or risks.

1. **Split before fitting the pre-processing.** Fitting the imputer and scaler on all rows lets validation and test statistics into training. The printed check shows this is not a no-op — a scaler mean moves by a quarter of a standard deviation (`field_size`, because recent seasons have 20 cars) — but the scores barely move, because a shifted mean changes the model's inputs by a constant that the model absorbs. The order is fixed as *split first* on principle: the leak is real even when its effect is small, it would grow with any step that uses the target, and avoiding it costs nothing.
2. **Feature engineering before the split.** If form features are built inside each split, every split starts with no memory: at the first validation race every driver looks like a rookie and the standings carried over from the previous season are gone. The printed check shows that career and circuit-history features change for most validation rows. The effect on the scores is nevertheless small, for a reason that is itself informative: the features that are damaged (career totals, circuit history) are the weak ones, while the strong ones (grid, qualifying, five-race form, in-season standings) either do not depend on history or rebuild within a few races. Building on the full chronological history and *then* splitting is kept because it is the correct description of what is known at race time, and it is leakage-safe since every feature only looks backwards (tested in Section 6.3). The general lesson: "split first" is not a universal rule — what matters is that no step uses information from the future of the row it computes.
3. **Impute before scaling.** The two orders give different transformed values for features with many missing entries (scaling first computes the standard deviation from observed values only; imputing first adds a block of identical medians and shrinks it), but the scores are unchanged. *Impute → scale* is kept because it is the conventional order and leaves every feature with exactly unit variance.

---

10. Models

All models are trained on the **same rows** (the training seasons of the modeling table) with the **same 28 features** and the same pre-processing, so their scores are directly comparable.

Model	Role
Grid-frequency rule	Reference with no machine learning: "how often did this grid slot reach the podium in the past?"
Logistic regression	**Statistical baseline** — the simplest reasonable model
Random forest	Statistical model 2
Gradient boosting	Statistical model 3
Shallow neural network	1–2 hidden layers, PyTorch

10.1 Metrics and decision threshold

Metric	What it tells us here	Role
PR-AUC (average precision)	How well the model ranks real podium finishers above the rest, focusing on the rare positive class	**Primary**
F1 at a chosen threshold	Balance of precision and recall once probabilities are turned into yes/no predictions	**Secondary**
ROC-AUC	Overall separation of the two classes	Reported
Precision, Recall	The two components of F1: "of the predicted podiums, how many were right" / "of the real podiums, how many were found"	Reported
Brier score, Accuracy	Quality of the probabilities; share of correct yes/no predictions	Reported for completeness

Why PR-AUC is the primary metric. Only about 15% of rows are podiums. ROC-AUC rewards a model for correctly ranking the ~85% of easy negatives (cars at the back of the grid almost never reach the podium), so every reasonable model scores above 0.9 and differences are compressed. PR-AUC ignores true negatives and measures what matters for this task: when the model says "podium", is it right, and does it find the real ones? A random model scores about 0.15 on PR-AUC (the positive share), so the scale is honest. PR-AUC also does not depend on a threshold, which makes it suitable for selecting hyperparameters.

Why F1 is the secondary metric. A user needs a decision, not only a ranking. F1 summarises the quality of the yes/no predictions without being inflated by the majority class, unlike accuracy — a model that always answers "no podium" has 85% accuracy and an F1 of 0.

Threshold. For each model the probability threshold is the one that maximises F1 **on the validation seasons**. It is then frozen and applied to the test seasons. The default 0.5 is not used because, with a rare positive class, the F1-optimal threshold is usually lower.

10.2 Reference points: a rule with no machine learning, and the statistical baseline

Grid-frequency rule. For each grid slot, the predicted probability is the share of drivers who reached the podium from that slot in the training seasons. This uses one feature and no learning algorithm. Any model has to beat it to justify its complexity.

Logistic regression (baseline). *How it works:* it computes a weighted sum of the standardised features and passes it through the logistic (S-shaped) function to obtain a probability. The weights are chosen to maximise the likelihood of the training labels, with an L2 penalty (strength `1/C`) that shrinks weights towards zero to limit over-fitting. Each weight has a direct reading: the change in the log-odds of a podium per one standard deviation of the feature, other features held fixed.

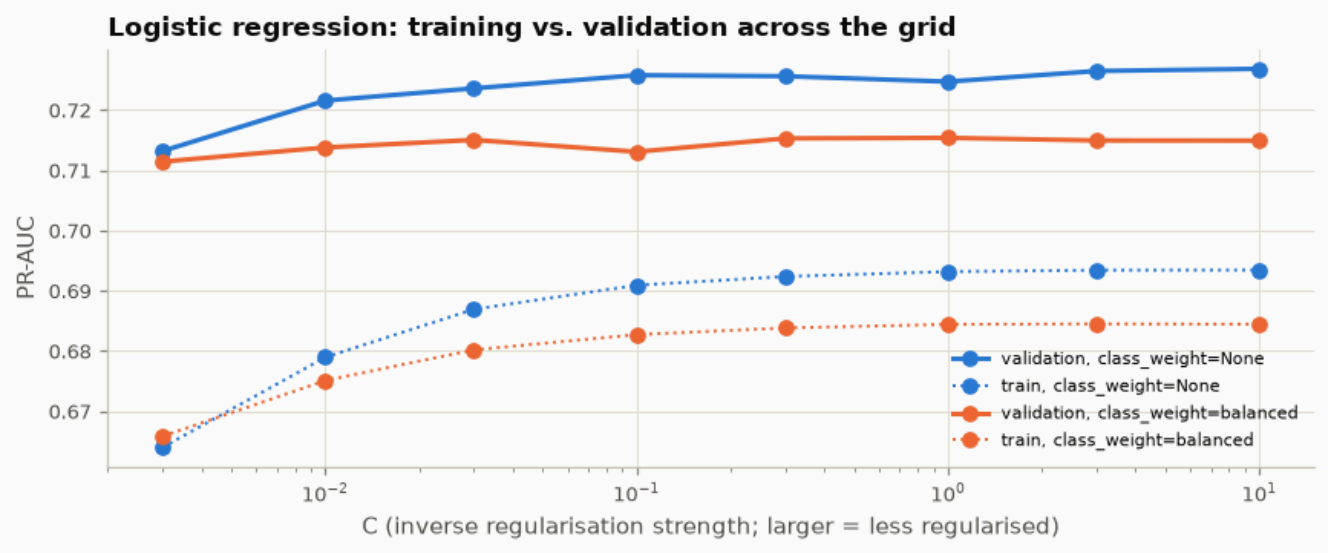
Limitations for this task: (1) each feature acts linearly on the log-odds, while Section 8.2 showed strongly curved relationships; (2) it has no interactions unless we add them — it cannot learn, for example, that team form matters more for a driver starting at the front; (3) with highly correlated features (Section 8.3) the individual weights are unstable and should not be over-interpreted; (4) it scores each row independently and does not know that exactly three drivers per race are positive. The last point applies to every model in this notebook.

```
Grid-frequency rule (no ML)               validation PR-AUC 0.6775   ROC-AUC 0.9217
Logistic regression (baseline, default)   validation PR-AUC 0.7247   ROC-AUC 0.9309
```

10.3 Logistic regression — tuned

Two settings are varied: the regularisation strength `C` (smaller = stronger penalty) and whether the classes are re-weighted (`class_weight="balanced"` makes the rare podium rows count more). Every combination is trained on the training seasons and scored on the validation seasons.

```
Selected on validation: {'C': 10.0, 'class_weight': None} -> val PR-AUC 0.7268
```



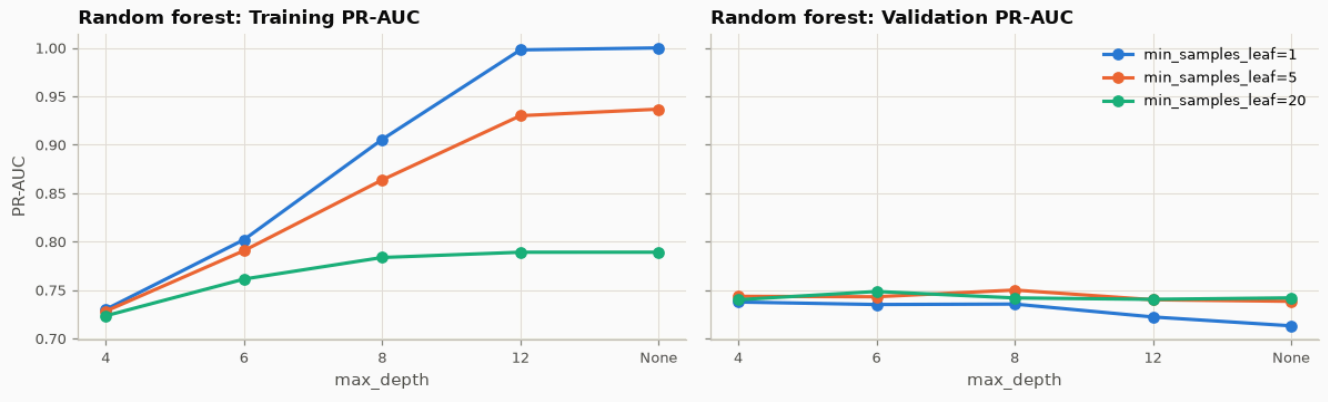
10.4 Random forest

How it works: it grows several hundred decision trees. Each tree is trained on a random resample of the training rows and, at every split, may only choose among a random subset of the features. A tree splits the data by thresholds (`grid_pos <= 3?`), so it handles curved relationships and interactions naturally. The forest's probability is the average over all trees; averaging many de-correlated trees reduces the variance of a single tree.

Limitations for this task: (1) a forest cannot extrapolate — it can only output averages of training outcomes, which matters when a new era produces feature combinations it has not seen; (2) probabilities are averaged votes and tend to be pulled away from 0 and 1; (3) with correlated features the importance is split arbitrarily between them; (4) it is far less transparent than a linear model, hence Section 13.

Tuned: tree depth (`max_depth`) and the minimum number of rows in a leaf (`min_samples_leaf`). Both limit how finely a tree may cut the data; larger leaves give smoother, less over-fitted probabilities.

```
Selected on validation: {'max_depth': 8, 'min_samples_leaf': 5} -> val PR-AUC 0.7500
```



The left panel shows the forest memorising the training data as trees get deeper and leaves smaller (training PR-AUC approaches 1), while the right panel shows that validation performance does not follow. The gap between the two is over-fitting; the selected setting is the one with the best *validation* score.

10.5 Gradient boosting

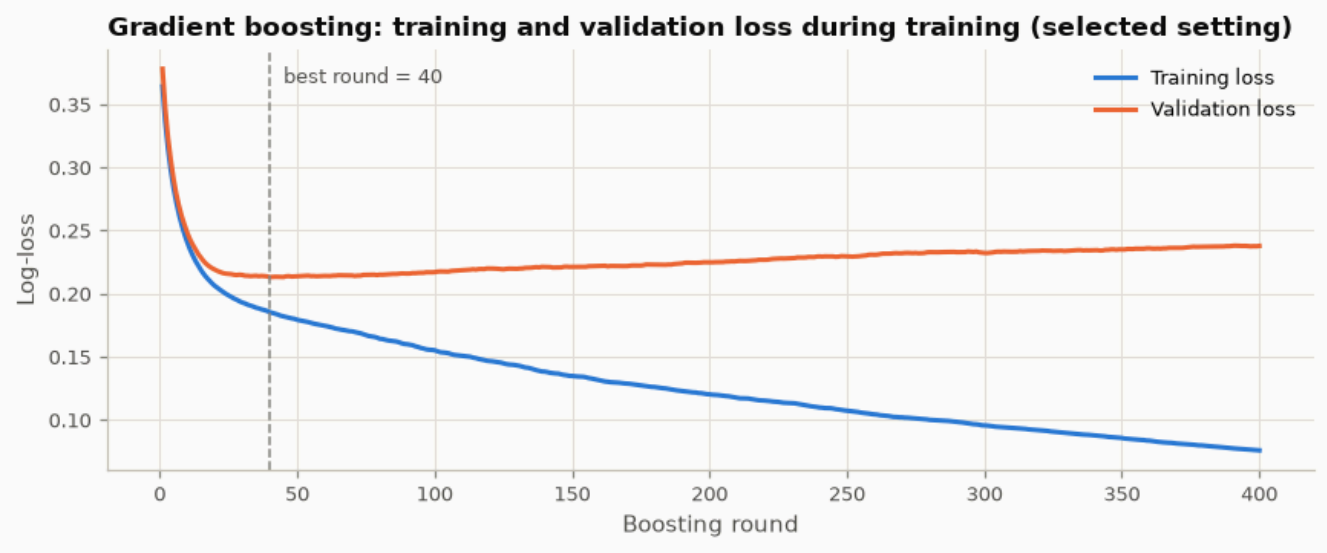
How it works: trees are built one after another. Each new, small tree is fitted to the errors (gradients of the log-loss) left by the trees so far, and its contribution is scaled down by the learning rate. The final score is the sum over all trees, converted to a probability. Where a forest reduces variance by averaging independent trees, boosting reduces bias by correcting mistakes step by step. The histogram-based implementation bins each feature first, which makes training fast.

Limitations for this task: (1) it over-fits if run for too many rounds, so the number of trees must be chosen on validation data; (2) it is sensitive to its settings (learning rate, depth); (3) like the forest it cannot extrapolate beyond the training range and is not directly interpretable.

Tuned: learning rate, tree depth, L2 regularisation, and the number of boosting rounds. For each setting the validation loss is tracked after every round, and the round with the lowest validation loss is kept.

```
Selected on validation: {'learning_rate': 0.1, 'max_depth': 4, 'l2_regularization': 1.0, 'max_iter':
40} -> val PR-AUC 0.7553
```

	learning_rate	max_depth	l2	rounds	train PR-AUC	val PR-AUC
0	0.10	4	1.0	40	0.7756	0.7553
1	0.03	3	1.0	214	0.7651	0.7515
2	0.10	4	0.0	38	0.7784	0.7513
3	0.03	2	0.0	203	0.7296	0.7504
4	0.10	3	1.0	42	0.7415	0.7498
5	0.03	3	0.0	205	0.7640	0.7488
6	0.03	2	1.0	255	0.7360	0.7477
7	0.10	2	0.0	95	0.7430	0.7475
8	0.03	4	0.0	148	0.7868	0.7468
9	0.10	3	0.0	61	0.7629	0.7459
10	0.10	2	1.0	70	0.7302	0.7459
11	0.03	4	1.0	151	0.7821	0.7454



The training loss keeps falling with every round, while the validation loss reaches a minimum and then rises: beyond that point the model is fitting noise in the training seasons. The model is stopped at the validation minimum.

10.6 Shallow neural network

Architecture. A feed-forward network: 28 inputs → one or two hidden layers with ReLU activation and dropout → one output unit whose sigmoid is the podium probability. "Shallow" means at most two hidden layers.

How it works: each hidden unit computes a weighted sum of its inputs and applies ReLU (`max(0, x)`), which lets the network bend straight lines into curves and combine features. The weights are learned by gradient descent (Adam optimiser) on the binary cross-entropy loss, in mini-batches of 64 rows.

Regularisation — three mechanisms, because the data set is small (about 6,400 training rows) for a neural network: **dropout** randomly switches off hidden units during training so the network cannot rely on any single unit; **weight decay** penalises large weights (the same idea as the L2 penalty in logistic regression); **early stopping** ends training when the validation loss has not improved for 15 epochs and restores the best epoch.

Limitations for this task: (1) tabular data with a few thousand rows is where neural networks have the least advantage over tree ensembles; (2) results vary with the random initialisation, so the spread over seeds is reported; (3) it needs scaled inputs and more tuning than the other models; (4) it is the least interpretable of the models here.

Tuned: the size and number of hidden layers, the dropout rate and the weight decay.

```
Selected on validation: {'hidden': (64,), 'dropout': 0.0, 'weight_decay': 0.0001} -> val PR-AUC 0.7296
(11s for 16 networks)
```

	hidden	dropout	weight_decay	best_epoch	train PR-AUC	val PR-AUC
0	(64,)	0.0	0.0001	35	0.7300	0.7296
1	(64, 32)	0.3	0.0001	20	0.7200	0.7291
2	(16,)	0.3	0.0001	38	0.7067	0.7261
3	(64,)	0.3	0.0001	37	0.7235	0.7255
4	(16,)	0.0	0.0001	34	0.7069	0.7253
5	(32, 16)	0.0	0.0001	22	0.7192	0.7217
6	(32, 16)	0.3	0.0001	23	0.7090	0.7188
7	(64, 32)	0.0	0.0100	32	0.7017	0.7172
8	(32, 16)	0.0	0.0100	32	0.6996	0.7154
9	(64, 32)	0.3	0.0100	21	0.6851	0.7149
10	(16,)	0.3	0.0100	47	0.6863	0.7131
11	(64, 32)	0.0	0.0001	10	0.7108	0.7124
12	(32, 16)	0.3	0.0100	29	0.6848	0.7112
13	(64,)	0.0	0.0100	38	0.6926	0.7092
14	(64,)	0.3	0.0100	31	0.6839	0.7092
15	(16,)	0.0	0.0100	51	0.6951	0.7064




```
Selected network over 5 random seeds: validation PR-AUC mean 0.7159, sd 0.0062, range 0.7073–0.7244
```

Reading the curves. The dashed line marks the epoch that early stopping kept. The right-hand panel shows the largest network with no dropout: its training loss keeps falling while the validation loss flattens and turns upward — the network starts memorising the training seasons, and it does so within a few epochs. Early stopping is what prevents this from reaching the final model; the grid above shows which combination of size, dropout and weight decay generalised best.

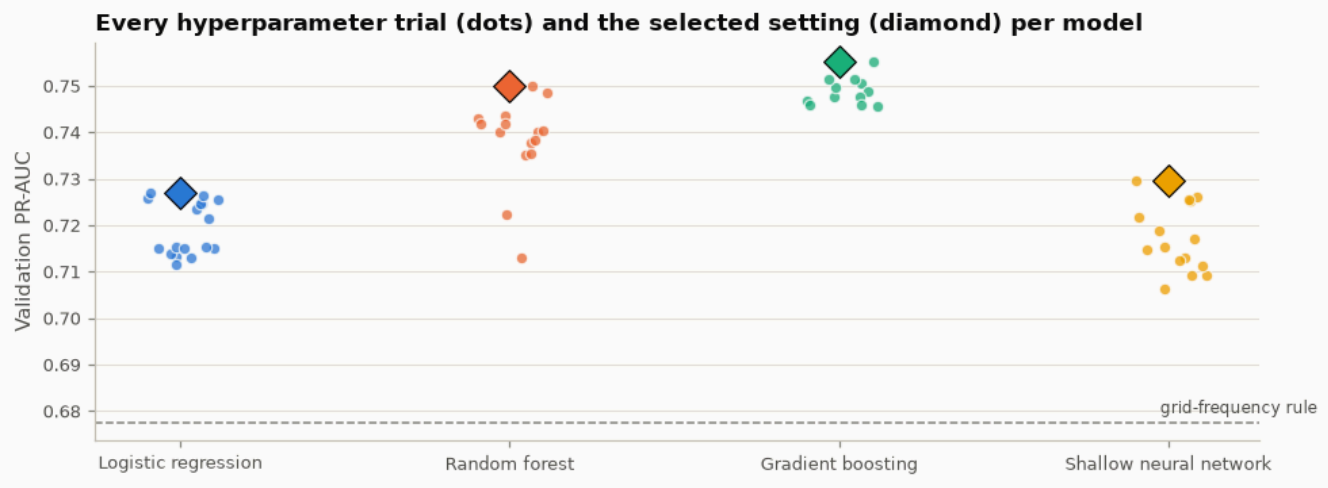
The spread over random seeds printed above is the yardstick for all neural-network comparisons that follow: a difference smaller than that spread is not meaningful.

Because the validation seasons are used both to stop training and to choose the architecture, the network's validation score is slightly optimistic. The test seasons, which play no part in either, give the unbiased figure (Section 11).

10.7 All modelling attempts

```
Total modelling attempts logged: 60
```

	attempts	worst_val	best_val	baseline_val
model				
Gradient boosting	12	0.7454	0.7553	0.7247
Logistic regression	17	0.7114	0.7268	0.7247
Random forest	15	0.7130	0.7500	0.7247
Shallow neural network	16	0.7064	0.7296	0.7247



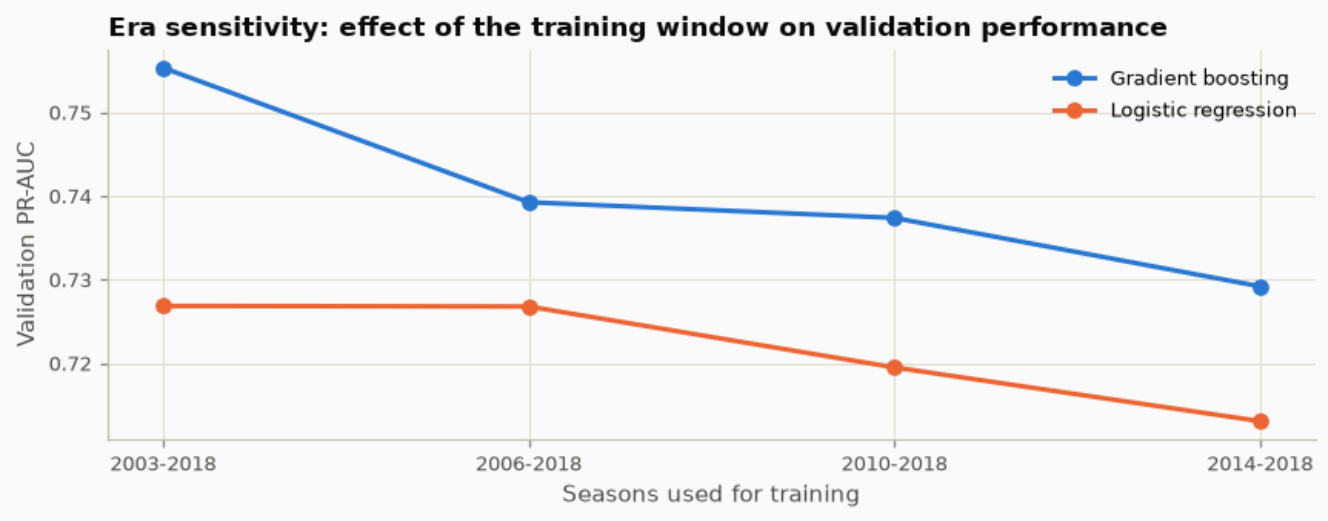
10.8 Two checks on the experimental design

Check 1 — chronological vs. random split. Section 7 argued for a chronological split. Here the same two models are also evaluated with a random row split of the same sizes, to see how different the reported score would have been.

Check 2 — era sensitivity. How much does the choice of training seasons matter? The models are re-trained on shorter, more recent windows and scored on the same validation seasons.

	chronological split (2019-2021)	random split, mean of 5	random split, sd	difference
Gradient boosting	0.7553	0.6914	0.0483	-0.0640
Logistic regression	0.7268	0.6777	0.0420	-0.0491

	model	Gradient boosting	Logistic regression
training seasons	**rows**		
2003-2018	**6411**	0.7553	0.7268
2006-2018	**5355**	0.7392	0.7268
2010-2018	**3877**	0.7374	0.7195
2014-2018	**2067**	0.7292	0.7131



Check 1 — split. With a random split the validation PR-AUC is 0.049 lower for logistic regression and 0.064 lower for gradient boosting than with the chronological split (random-split standard deviation about 0.045). Two things differ at once, so this is not a clean measurement of leakage: a random validation set is drawn from all of 2003–2021 and is therefore a different, more mixed set of races than 2019–2021, and its score swings widely from one random draw to the next. The two schemes also do not ask the same question: a random split asks "how well does the model fill in races *between* races it has seen?", the chronological split asks "how well does it predict seasons it has *never* seen?". Only the second question matches real use, so the chronological figures are the ones reported, whichever is larger.

Check 2 — era. Across training windows from 2003–2018 down to 2014–2018, validation PR-AUC varies by 0.014 (logistic regression) and 0.026 (gradient boosting). Dropping older seasons removes up to two thirds of the training rows; the table shows what that trade — fewer but more recent rows — does to performance. The full 2003–2018 window is kept for the final models because it uses all available data with complete qualifying coverage and the features are built to be era-robust (shares and rates instead of raw points).

11. Model comparison

The selected setting of each model is now evaluated on all three sets. The decision threshold of each model is chosen on the validation seasons and applied unchanged to the test seasons. **This is the first and only time the test seasons are used.**

```
TRAINING seasons 2003–2018
```

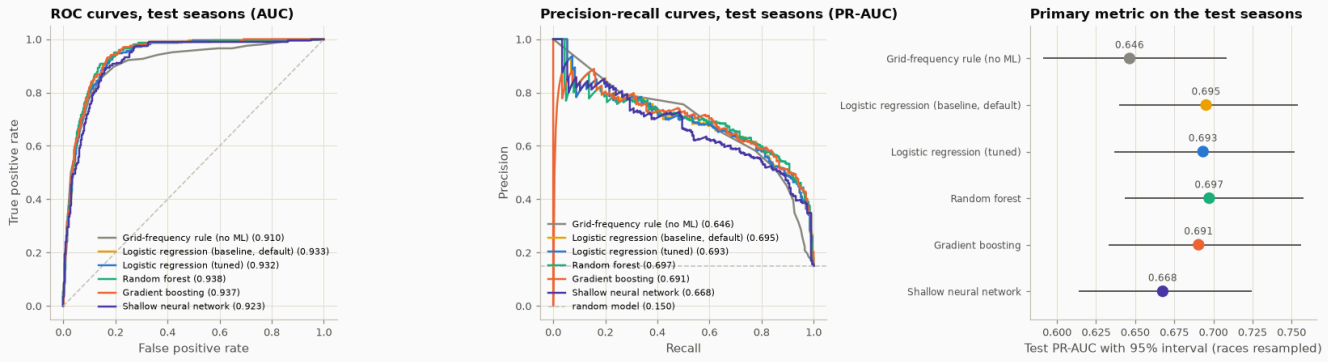
	threshold	ROC-AUC	PR-AUC	F1	Precision	Recall	Brier	Accuracy
model								
Grid-frequency rule (no ML)	0.500	0.909	0.618	0.638	0.638	0.638	0.072	0.898
Logistic regression (baseline, default)	0.296	0.932	0.693	0.667	0.585	0.777	0.067	0.891
Logistic regression (tuned)	0.295	0.932	0.693	0.669	0.585	0.781	0.067	0.892
Random forest	0.387	0.973	0.864	0.776	0.744	0.812	0.047	0.934
Gradient boosting	0.341	0.956	0.776	0.717	0.650	0.799	0.058	0.911
Shallow neural network	0.445	0.946	0.730	0.685	0.694	0.676	0.062	0.913

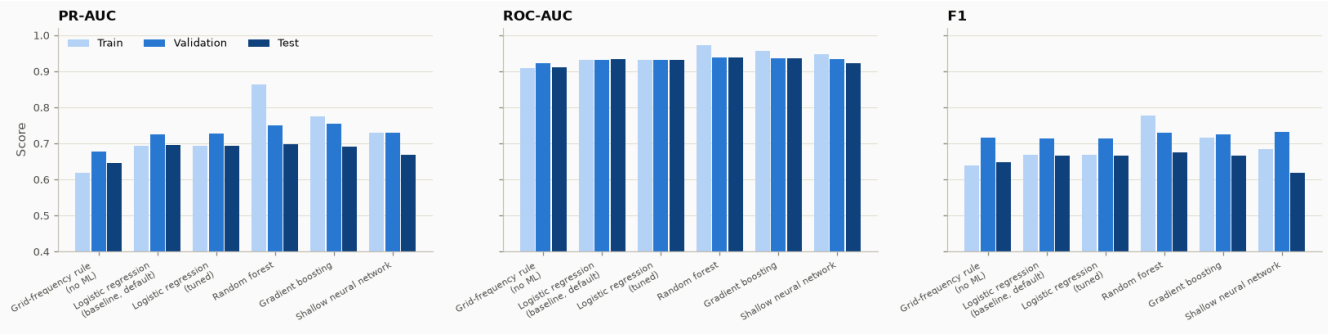
```
VALIDATION seasons 2019–2021
```

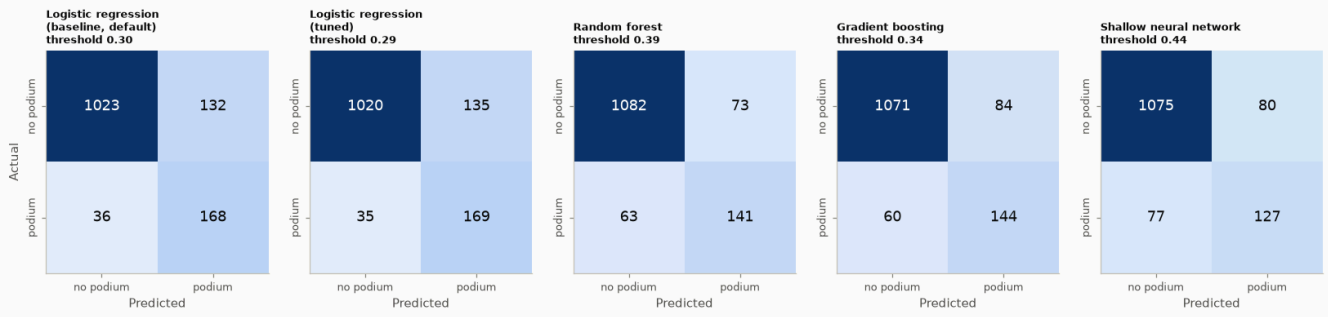
	threshold	ROC-AUC	PR-AUC	F1	Precision	Recall	Brier	Accuracy
model								
Grid-frequency rule (no ML)	0.500	0.922	0.678	0.717	0.717	0.717	0.067	0.915
Logistic regression (baseline, default)	0.296	0.931	0.725	0.714	0.634	0.817	0.066	0.902
Logistic regression (tuned)	0.295	0.931	0.727	0.714	0.634	0.817	0.066	0.902
Random forest	0.387	0.939	0.750	0.730	0.697	0.767	0.063	0.915
Gradient boosting	0.341	0.937	0.755	0.724	0.665	0.794	0.063	0.909
Shallow neural network	0.445	0.933	0.730	0.732	0.714	0.750	0.064	0.917

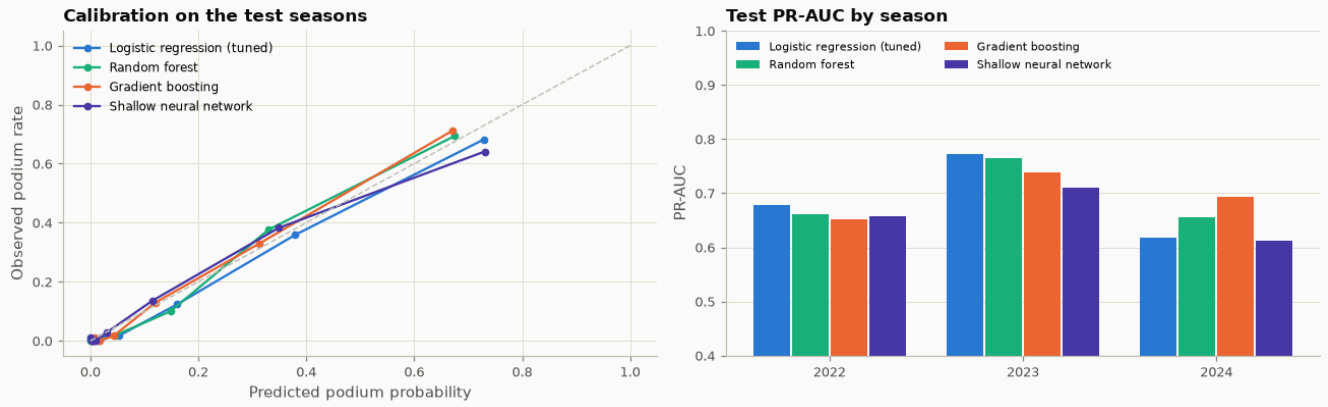
```
TEST seasons 2022–2024 (unseen)
```

	threshold	ROC-AUC	PR-AUC	F1	Precision	Recall	Brier	Accuracy
model								
Grid-frequency rule (no ML)	0.500	0.910	0.646	0.649	0.650	0.647	0.073	0.895
Logistic regression (baseline, default)	0.296	0.933	0.695	0.667	0.560	0.824	0.071	0.876
Logistic regression (tuned)	0.295	0.932	0.693	0.665	0.556	0.828	0.072	0.875
Random forest	0.387	0.938	0.697	0.675	0.659	0.691	0.070	0.900
Gradient boosting	0.341	0.937	0.691	0.667	0.632	0.706	0.070	0.894
Shallow neural network	0.445	0.923	0.668	0.618	0.614	0.623	0.076	0.884









model	Logistic regression (tuned)	Random forest	Gradient boosting	Shallow neural network
season				
2022	0.678	0.662	0.652	0.657
2023	0.773	0.765	0.739	0.710
2024	0.618	0.657	0.693	0.612

11.1 A decision rule that uses the structure of a race

Every model scores each driver on its own, but a race has **exactly three** podium places. An alternative to a probability threshold is therefore: *in each race, predict the three drivers with the highest probability.* This needs no threshold at all and uses only the fact that the rows of a race belong together, which is known before the race.

It is reported as an additional result, next to the threshold rule.

	val F1 (threshold)	val F1 (top-3 per race)	test F1 (threshold)	test F1 (top-3 per race)
model				
Grid-frequency rule (no ML)	0.717	0.717	0.649	0.652
Logistic regression (baseline, default)	0.714	0.683	0.667	0.647
Logistic regression (tuned)	0.714	0.689	0.665	0.642
Random forest	0.730	0.700	0.675	0.647
Gradient boosting	0.724	0.711	0.667	0.652
Shallow neural network	0.732	0.700	0.618	0.608

11.2 What the comparison shows

Selection (validation seasons only). Best statistical model: **Gradient boosting** (validation PR-AUC 0.755). Best model overall: **Gradient boosting** (validation PR-AUC 0.755). This is the model used for the inference function.

Performance on the unseen test seasons (2022–2024).

Model	PR-AUC (95% interval)	ROC-AUC	F1	Precision	Recall
Grid-frequency rule (no ML)	0.646 (0.591–0.709)	0.910	0.649	0.650	0.647
Logistic regression (baseline, default)	0.695 (0.640–0.754)	0.933	0.667	0.560	0.824
Logistic regression (tuned)	0.693 (0.637–0.752)	0.932	0.665	0.556	0.828
Random forest	0.697 (0.644–0.758)	0.938	0.675	0.659	0.691
Gradient boosting	0.691 (0.633–0.756)	0.937	0.667	0.632	0.706
Shallow neural network	0.668 (0.614–0.725)	0.923	0.618	0.614	0.623

Against the baselines.

- The rule with no machine learning (grid slot only) already reaches a test PR-AUC of 0.646. Starting position explains most of what can be predicted.
- The logistic-regression baseline reaches 0.695: adding qualifying, form and standings features is worth +0.049 PR-AUC over the grid rule.
- The selected model, Gradient boosting, reaches 0.691, i.e. -0.005 relative to the logistic baseline. The baseline's score lies inside the selected model's 95% interval, so the extra complexity is not clearly rewarded on these 68 test races.
- The shallow neural network reaches 0.668 (-0.028 vs. the logistic baseline; its seed-to-seed standard deviation on validation was 0.006).
- The highest test PR-AUC belongs to Random forest. The model was **not** chosen on this number; it is reported for transparency.

Over-fitting. The gap between training and test PR-AUC is +0.000 for Logistic regression (tuned), +0.166 for Random forest, +0.085 for Gradient boosting, +0.062 for Shallow neural network. A large gap means the model fits the training seasons much better than it generalises.

Decision rule. Predicting the top three per race instead of using a threshold changes test F1 by -0.023 (Logistic regression (tuned)), -0.028 (Random forest), -0.015 (Gradient boosting), -0.010 (Shallow neural network).

Interpretation.

- **The models are close to each other, and all are far above chance.** The intervals in the right-hand panel of the curves figure overlap heavily. With 68 test races, differences of one or two hundredths in PR-AUC between the learned models should not be read as one model being truly better. What is clear is the ordering *random (≈0.15) ≪ grid rule < models with the full feature set*.
- **Why a simple model does so well.** Most of the predictable signal is in a handful of strong, nearly monotonic features (grid, qualifying gap, team form). A linear model on standardised inputs captures a monotonic effect well, so the room left for more flexible models is small. The remaining errors are mostly events no pre-race feature can see: first-lap collisions, mechanical failures, safety cars, weather and strategy.
- **Why more flexible models do not automatically win here.** The training set is small (6,411 rows, 900 positives) and the test seasons come from a new regulation era. Flexible models can fit era-specific patterns in the training seasons that do not carry over, which shows up as a larger train–test gap.
- **Validation vs. test.** Scores differ between the validation and test seasons for every model, including the grid rule that has nothing to tune. That points to the seasons themselves being more or less predictable (for example, how dominant the leading team was), not to over-fitting of the validation set. The per-season panel shows this variation directly.
- **Calibration.** The calibration panel shows whether a predicted probability can be read as a frequency. Points near the diagonal mean that "70%" really happens about 70% of the time.
- **Baseline requirement.** The baseline is logistic regression with default settings. The more complex models are justified only to the extent that they beat it on validation; where they do not, the table says so.

---

12. Experiment: feature-group ablation

Section 8 justified the feature groups **before** training. The ablation measures what each group actually contributes **after** training.

Method. One group is removed, every model is re-trained from scratch with its selected hyperparameters on the remaining features, and the score is compared with the full model. This is repeated for each of the six groups. The neural network is trained with three random seeds per configuration and averaged, because its seed-to-seed variation is of the same size as some of the effects being measured.

A second view, **"only this group"**, trains each model on a single group. The two views answer different questions:

- *Drop one group* → the group's **unique** contribution: what is lost that no other group can replace.
- *Only this group* → the group's **stand-alone** value.

Because the groups overlap heavily (Section 8.3), a group can be very informative on its own and still cause almost no loss when dropped.

```
52 configurations trained in 32s
Full models (reference):
```

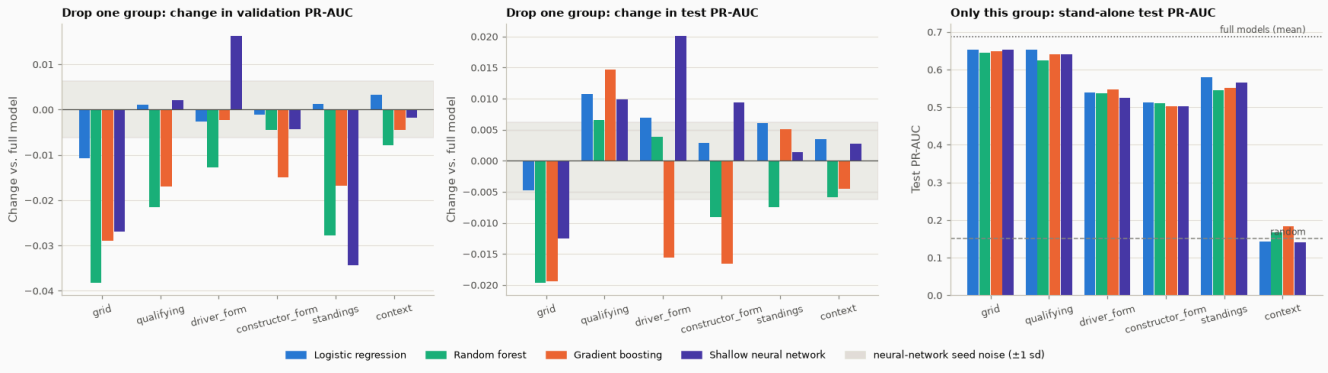
	val PR-AUC	test PR-AUC	val ROC-AUC	test ROC-AUC	test F1
model					
Logistic regression	0.7268	0.6934	0.9307	0.9323	0.6654
Random forest	0.7500	0.6973	0.9387	0.9378	0.6746
Gradient boosting	0.7553	0.6906	0.9368	0.9370	0.6667
Shallow neural network	0.7241	0.6682	0.9331	0.9238	0.6204

```
Change in score when one group is REMOVED (negative = the model got worse without it):
```

	Δ val PR-AUC				Δ test PR-AUC			
model	**Gradient boosting**	**Logistic regression**	**Random forest**	**Shallow neural network**	**Gradient boosting**	**Logistic regression**	**Random forest**	**Shallow neural network**
group								
grid	-0.0289	-0.0108	-0.0383	-0.0269	-0.0194	-0.0047	-0.0197	-0.0125
qualifying	-0.0171	0.0010	-0.0216	0.0021	0.0147	0.0108	0.0065	0.0099
driver_form	-0.0023	-0.0027	-0.0128	0.0162	-0.0156	0.0069	0.0039	0.0201
constructor_form	-0.0150	-0.0011	-0.0044	-0.0043	-0.0165	0.0028	-0.0091	0.0094
standings	-0.0169	0.0012	-0.0277	-0.0344	0.0050	0.0061	-0.0075	0.0014
context	-0.0045	0.0032	-0.0079	-0.0018	-0.0045	0.0035	-0.0059	0.0028

Change in ROC-AUC and F1 on the test seasons:

	Δ test ROC-AUC				Δ test F1			
model	**Gradient boosting**	**Logistic regression**	**Random forest**	**Shallow neural network**	**Gradient boosting**	**Logistic regression**	**Random forest**	**Shallow neural network**
group								
grid	-0.0012	-0.0027	-0.0030	-0.0033	0.0058	0.0013	-0.0309	-0.0146
qualifying	0.0015	0.0038	-0.0017	0.0047	-0.0068	-0.0037	-0.0340	0.0038
driver_form	-0.0046	0.0003	0.0011	0.0073	-0.0255	-0.0101	0.0075	0.0432
constructor_form	-0.0010	0.0007	0.0003	0.0034	-0.0015	0.0108	0.0071	0.0120
standings	0.0003	0.0014	-0.0008	0.0021	0.0133	-0.0040	0.0141	0.0053
context	0.0012	0.0004	0.0008	0.0036	-0.0100	0.0054	0.0030	0.0126



Ablation results (mean over the four models)

Group	Features	Drop it: Δ val PR-AUC	Drop it: Δ test PR-AUC	Only it: test PR-AUC
grid	3	-0.0262	-0.0141	0.649
qualifying	5	-0.0089	+0.0105	0.640
driver_form	7	-0.0004	+0.0038	0.537
constructor_form	3	-0.0062	-0.0034	0.507
standings	5	-0.0194	+0.0013	0.560
context	5	-0.0028	-0.0010	0.159

- **Largest unique contribution:** removing **grid** costs the most on validation (-0.0262 PR-AUC on average), followed by **standings** (-0.0194).

- **Strongest stand-alone group: grid** alone reaches a test PR-AUC of 0.649; the weakest alone is **context** (0.159, against 0.15 for a random model).
- **Groups whose removal hurts every model on both validation and test:** grid.

Interpretation.

- **The groups overlap, so drop-one effects are small.** Grid, qualifying, team form and standings all measure "how fast is this car–driver combination right now". Remove one, and the others cover most of the loss. That is why the *drop-one* effects are a few hundredths of PR-AUC at most, even for groups that are strong on their own in the *only-this-group* panel. A small drop-one effect does **not** mean a group is useless; it means it is replaceable.
- **Grid is the group with the clearest unique contribution.** Its removal lowers the score for the models on both validation and test (see the computed list above), even though qualifying position is still available. The reason is the one found in Question 1: the grid differs from the qualifying order in about a quarter of entries, and when it does, the actual starting slot is what counts.
- **Qualifying, standings and form are largely interchangeable.** Removing one of them hurts on validation for the tree models but not consistently on test. Where a group's removal helps on one set and hurts on the other, the honest reading is "no measurable unique contribution", not "harmful".
- **Effects inside the grey band are noise.** Changes smaller than the neural network's seed-to-seed standard deviation cannot be distinguished from re-running the same model. The same caution applies to the other models given the size of the validation and test sets.
- **Context** is the weakest group: alone it is at the level of a random model (three of its five features are identical for every driver in a race), and its drop-one effect is close to zero. It is kept because it costs five columns and the ablation gives no consistent evidence that it hurts; it is the first candidate for removal in a leaner model.
- **Relation to the pre-training evidence.** The stand-alone ranking of the groups matches the single-feature ranking of Section 8.1 (grid and qualifying first, context last), so the exploratory evidence and the post-training measurement tell the same story.

---

13. Model explainability

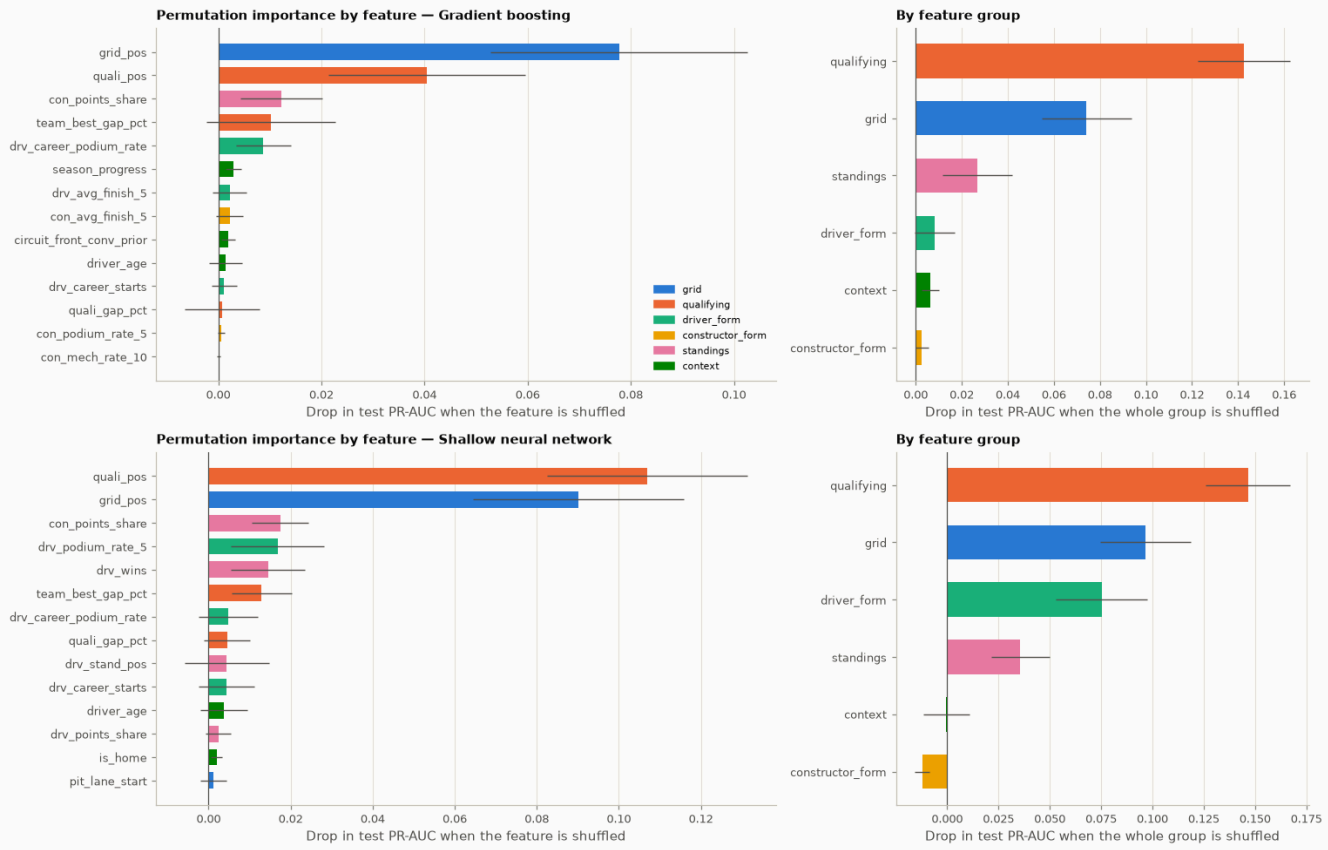
> **Explanations are not causal effects.** Everything in this section describes **what the model relies on** to make its predictions. A feature being important to the model is **not** proof that it *causes* podium finishes. The features are correlated with each other and with things the model never sees (the true pace of the car, driver skill, budget). For example, a high importance for championship standing does not mean that being higher in the standings makes a car faster; standing and podiums are both consequences of having a fast car and a good driver. The plots below answer "why did the model predict this?", never "what would happen if we changed this feature in the real world?".

Two models are explained: the best statistical model and the shallow neural network. Three methods are used:

Method	Scope	Idea
Permutation importance	Global	Shuffle one feature in unseen data and measure how much the score drops
SHAP	Global and local	Split each individual prediction into additive contributions of the features
LIME	Local	Fit a small linear model around one prediction to approximate the model nearby

13.1 Global: permutation importance

Each feature is shuffled 20 times in the **test** seasons and the drop in PR-AUC is recorded. The test set is used because importance on unseen data shows what the model needs in order to generalise, not what it memorised. Because correlated features can stand in for each other, a single shuffled feature can look unimportant while its *group* is essential. The right-hand panel therefore shuffles all features of a group **together**.



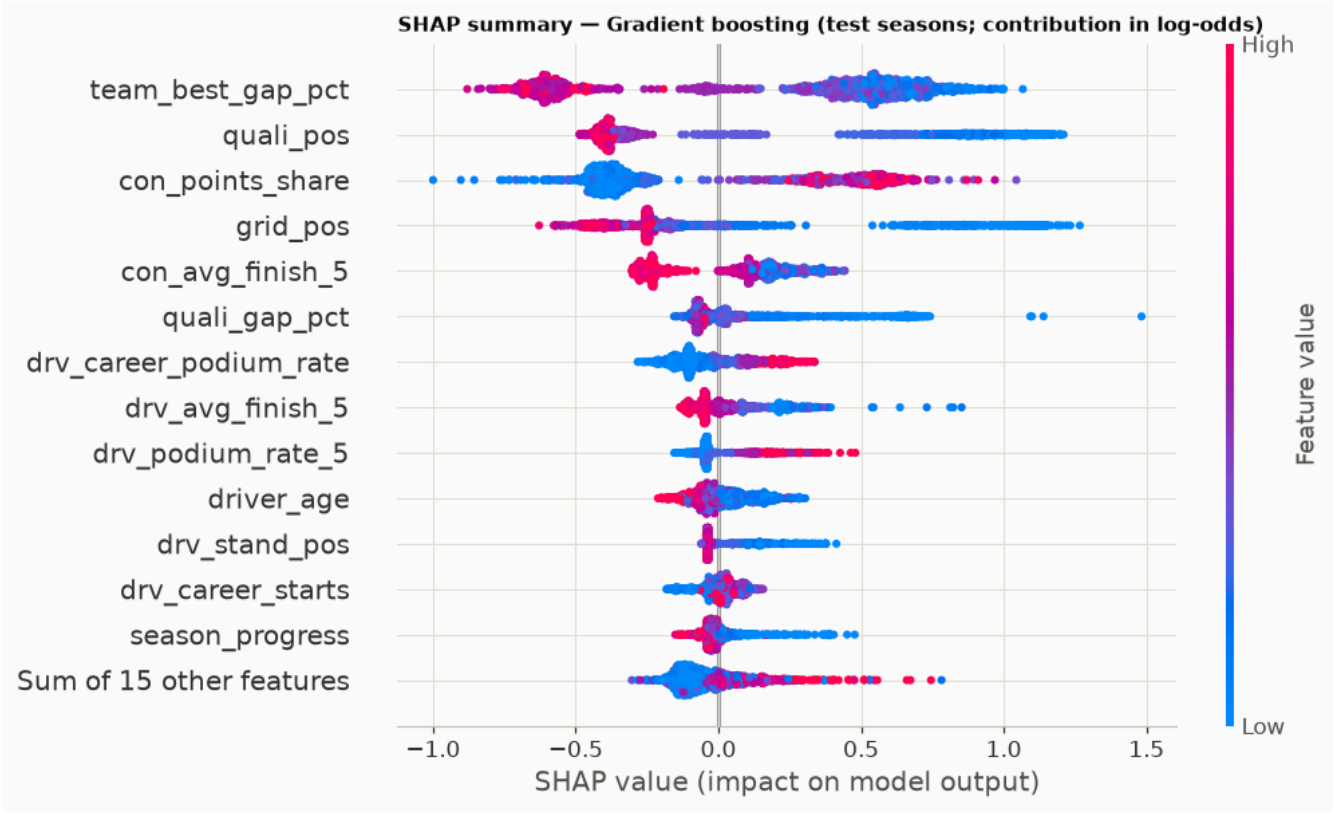
13.2 Global: SHAP

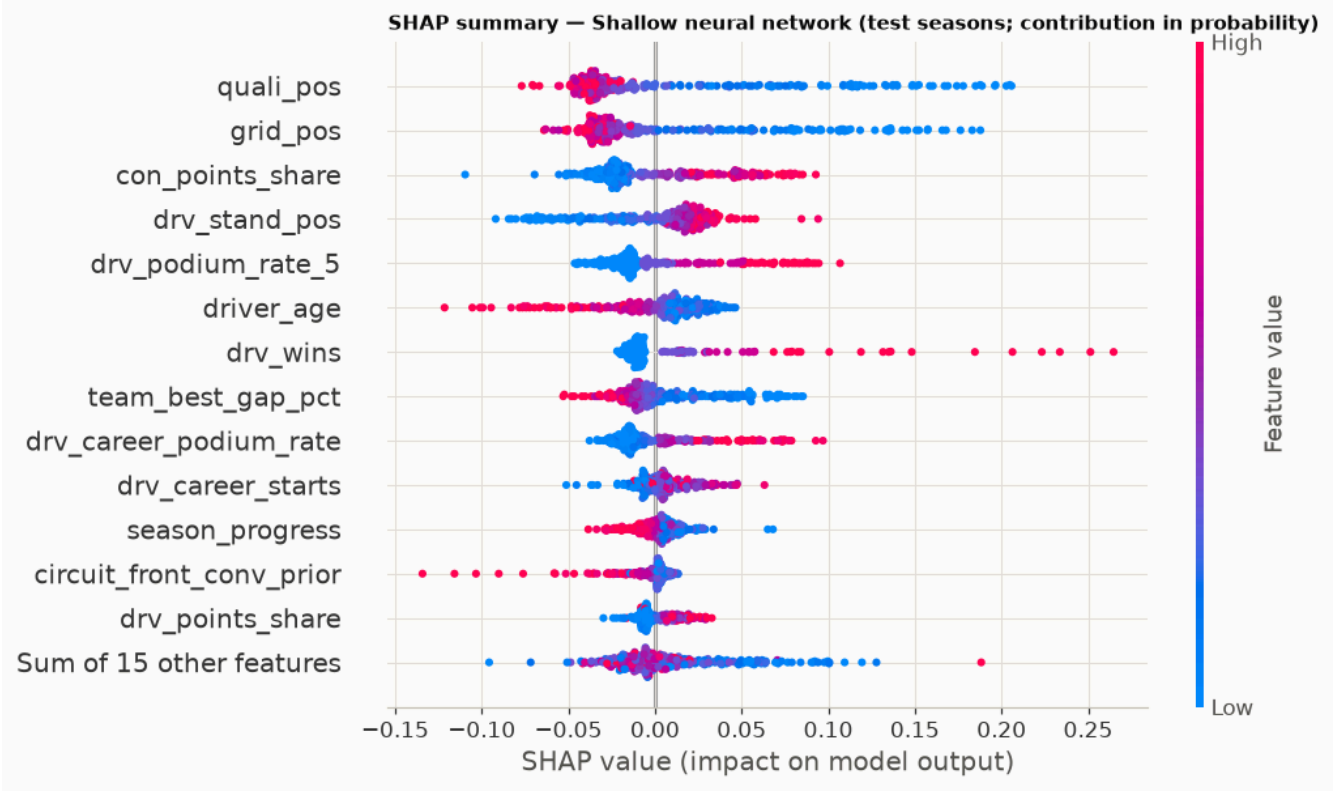
SHAP assigns to each feature, for each individual prediction, a contribution that says how far that feature pushed the prediction above or below the model's average output. The contributions of all features add up exactly to the prediction. Averaging their absolute size over many rows gives a global importance; plotting them against the feature value shows the *direction* of the effect, which permutation importance cannot show.

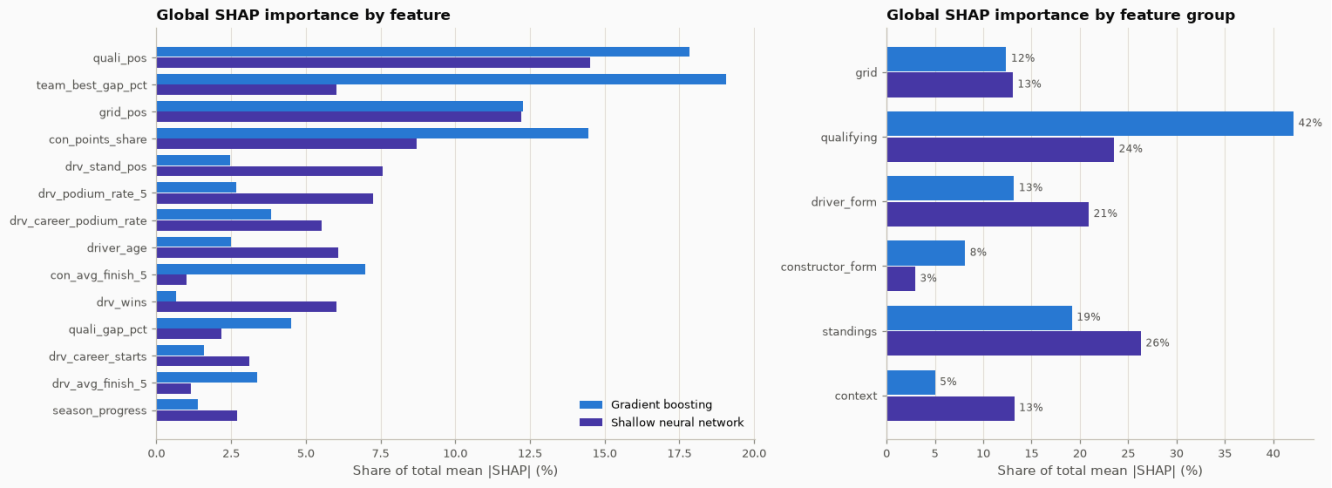
- For the statistical model an exact, model-specific explainer is used (tree explainer for the tree ensembles, linear explainer for logistic regression).
- For the neural network a model-agnostic explainer is used on a sample of rows, with a sample of training rows as the reference ("background") data.

Contributions are computed on the pre-processed inputs the model actually sees; the plots show the original feature values so they can be read in real units.

```
SHAP values computed in 3s
```


SHAP summary — Gradient boosting (test seasons; contribution in log-odds)
team_best_gap_pct
quali_pos
con_points_share
grid_pos
con_avg_finish_5
quali_gap_pct
drv_career_podium_rate
drv_avg_finish_5
drv_podium_rate_5
driver_age
drv_stand_pos
drv_career_starts
season_progress
Sum of 15 other features
−1.0
−0.5
0.0
0.5
1.0
1.5
SHAP value (impact on model output)
High
Feature value
Low


SHAP summary — Shallow neural network (test seasons; contribution in probability)
quali_pos
grid_pos
con_points_share
drv_stand_pos
drv_podium_rate_5
driver_age
drv_wins
team_best_gap_pct
drv_career_podium_rate
drv_career_starts
season_progress
circuit_front_conv_prior
drv_points_share
Sum of 15 other features
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15
0.20
0.25
SHAP value (impact on model output)
High
Feature value
Low



	Gradient boosting	Shallow neural network
grid	12.4	13.1
qualifying	42.1	23.5
driver_form	13.2	20.9
constructor_form	8.1	3.0
standings	19.2	26.3
context	5.0	13.2

Global explanations — what the models rely on

	Gradient boosting	Shallow neural network
Top features by permutation importance	`grid_pos`, `quali_pos`, `con_points_share`, `team_best_gap_pct`, `drv_career_podium_rate`	`quali_pos`, `grid_pos`, `con_points_share`, `drv_podium_rate_5`, `drv_wins`
Top features by SHAP	`team_best_gap_pct`, `quali_pos`, `con_points_share`, `grid_pos`, `con_avg_finish_5`	`quali_pos`, `grid_pos`, `con_points_share`, `drv_stand_pos`, `drv_podium_rate_5`
Top group, permutation	qualifying	qualifying
Top group, SHAP	qualifying (42%)	standings (26%)

For Gradient boosting, 7 of the top 8 features are the same under permutation importance and SHAP.

Interpretation of the global explanations.

- **Both models lean first on where the car starts and how fast it was in qualifying**, then on how the team and driver have been doing this season (standings and recent form). This agrees with the exploratory ranking in Section 8.1 and with the ablation in Section 12, so three independent pieces of evidence point the same way.
- **Direction (SHAP summary plots).** In the summary plots each dot is one driver–race; colour is the feature value. Low values of `grid_pos` and `quali_pos` (blue) lie on the right-hand side — a start near the front pushes the prediction towards "podium" — and high values of the form and points-share features (red) also lie on the right. The model has learned the directions that the EDA showed.
- **Shape.** For the position features the positive contributions reach much further from zero than the negative ones: starting at the very front raises the prediction a lot, while starting anywhere from mid-grid backwards lowers it by a similar, moderate amount. This reflects Section 8.2 — the podium rate is almost flat near zero from mid-grid backwards.

- **Why single-feature permutation importances are small.** Shuffling one feature usually costs little because a correlated feature still carries the same information (`grid_pos` and `quali_pos` have a correlation of 0.97). The group-level panels are the more trustworthy summary: shuffling a whole group removes that information completely.
- **Two methods, slightly different answers.** Permutation importance measures the loss in *ranking quality* when a feature is destroyed; SHAP measures how much a feature *moves the predictions*. A feature can move many predictions a little without changing the ranking much. Where the two disagree on the order of similar features, neither is "wrong"; the importance is being shared among near-duplicates.
- **Context features contribute little to ranking quality** (group permutation importance near zero), in line with the ablation, even where their SHAP share is not negligible: they move predictions slightly without improving the ranking.
- **Not causal.** These plots show reliance, not cause. "Grid position is the most important feature" means the model uses it most. It does not mean that moving a slow car to pole would give it the podium rate of a pole-sitter: cars are on pole *because* they are fast, and the model cannot separate the two.

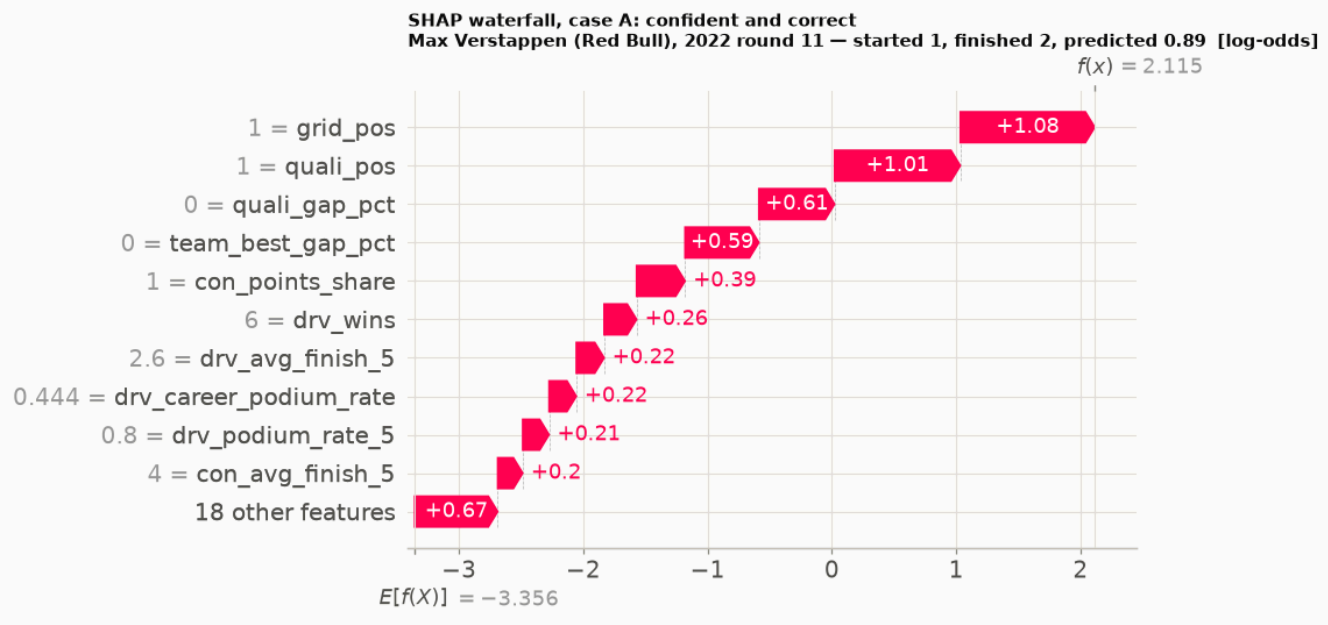
13.3 Local: explaining single predictions

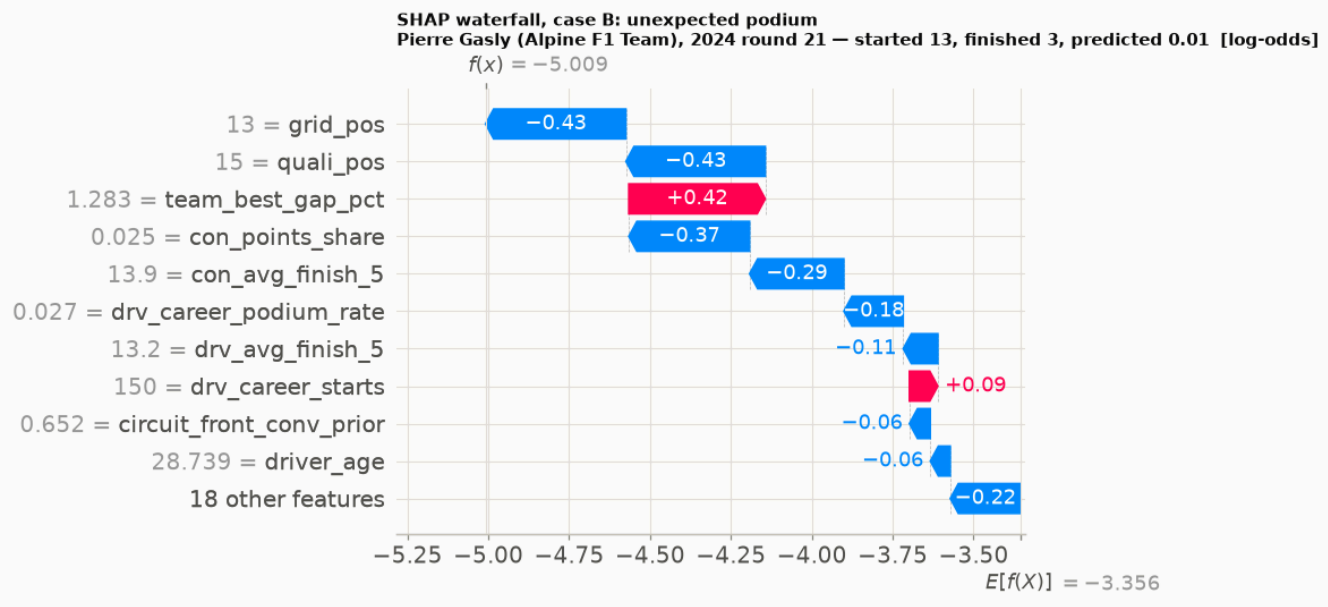
Two individual predictions from the test seasons are explained, chosen by fixed rules so the choice is not cherry-picked:

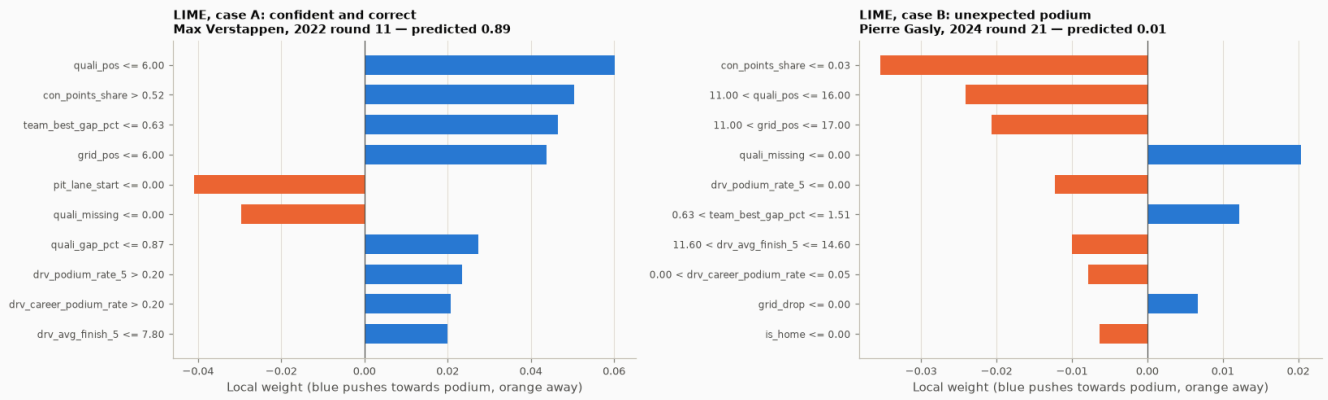
- **Case A — a confident, correct podium prediction:** the test row with the highest predicted probability that was a podium.
- **Case B — a podium the model did not expect:** the real podium finish with the *lowest* predicted probability. This shows what the model cannot see.

Each case is explained with a **SHAP waterfall** plot and with **LIME**, for the best statistical model.

	year	round	circuit	driver	constructor	grid	positionOrder	probability
case								
A: confident and correct	2022	11	Red Bull Ring	Max Verstappen	Red Bull	1	2	0.892
B: unexpected podium	2024	21	Autódromo José Carlos Pace	Pierre Gasly	Alpine F1 Team	13	3	0.007







	SHAP top 5	LIME top 5	shared	shared with same direction
case				
A: confident and correct	grid_pos, quali_pos, quali_gap_pct, team_best_gap_pct, con_points_share	quali_pos, con_points_share, team_best_gap_pct, grid_pos, pit_lane_start	4	4
B: unexpected podium	grid_pos, quali_pos, team_best_gap_pct, con_points_share, con_avg_finish_5	con_points_share, quali_pos, grid_pos, quali_missing, drv_podium_rate_5	3	3

Reading the two cases (model: Gradient boosting)

Case A — Max Verstappen (Red Bull), 2022 round 11, Red Bull Ring. Started 1, finished 2; predicted podium probability **0.89**. The features that pushed the prediction up the most were `grid_pos` = 1.00, `quali_pos` = 1.00, `quali_gap_pct` = 0.00. The strong signals all point the same way, which is why the model is so confident.

Case B — Pierre Gasly (Alpine F1 Team), 2024 round 21, Autódromo José Carlos Pace. Started 13, finished 3; predicted podium probability **0.01**. The features that pushed the prediction *down* the most were `grid_pos` = 13.00, `quali_pos` = 15.00, `con_points_share` = 0.02. On pre-race information this entry looked like a clear non-podium, and the model said so. The podium came from what happened *during* the race, which no pre-race feature can contain. This is not a defect of this model in particular; it marks the limit of what can be predicted at the cutoff.

SHAP vs. LIME. The two methods share 4 of their top five features in case A and 3 in case B. They are not expected to match exactly: SHAP distributes the exact difference between this prediction and the average prediction, while LIME fits an approximate linear model on random points around the row and reports rule-like

conditions (for example `grid_pos <= 6`). Agreement on the main features in both methods is a sign the explanation is robust; features that appear in only one method should be treated with caution. In particular, LIME can give weight to flags that are constant for almost every row (such as `pit_lane_start <= 0`); this is a side effect of how it samples around the row, not a real driver of the prediction.

Reminder. The local explanations say which inputs led *the model* to its output for that row. They do not say why the driver did or did not finish on the podium, and they do not say what would have happened had an input been different.

---

14. Inference function

`predict_podium` takes the pre-race feature values of one driver in one race and returns the podium probability and decision from the selected model.

- **Input:** a dictionary of feature names and values. Any feature may be left out or set to `None`.
- **Missing values:** filled by the model's own pipeline with the medians learned from the training seasons, exactly as during training. The output lists which features were filled, so the caller knows how much the prediction rests on defaults.
- **Validation:** unknown feature names raise an error, so a typing mistake cannot silently turn into a missing value.
- **Decision:** the threshold chosen on the validation seasons.

```
Inference model: Gradient boosting, decision threshold 0.341
```

14.1 Demonstration on a complete record

A real entry from the last race in the data is used: the driver who started from pole position.

```
Entry: Lando Norris (McLaren), 2024 Yas Marina Circuit — started 1, finished 1

Input record:
```

```
Output:
  podium_probability: 0.7519
  prediction: podium
  threshold: 0.3405
  model: Gradient boosting
  features_provided: 28
  features_imputed: []
```

14.2 Demonstration on a partially-missing record

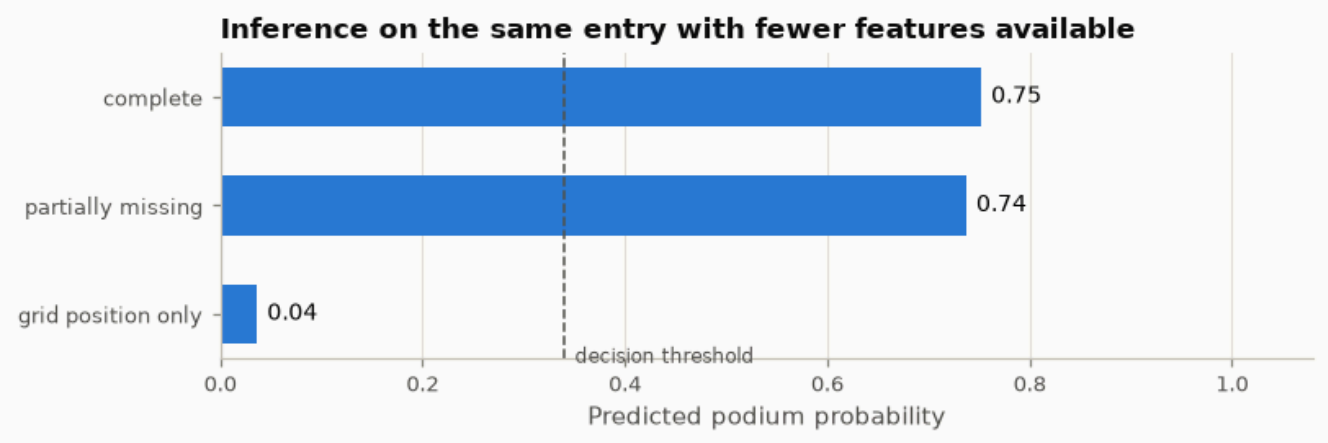
The same entry, but now assume the qualifying lap times were not delivered and nothing is known about the driver's history at this circuit: five features are removed. Grid position, standings and recent form are still provided.

```
Output:
  podium_probability: 0.7371
  prediction: podium
  threshold: 0.3405
  model: Gradient boosting
  features_provided: 22
  features_imputed: ['quali_gap_pct', 'mate_quali_gap_pct', 'team_best_gap_pct', 'drv_dnf_rate_10',
'drv_circuit_podium_rate', 'con_mech_rate_10']

With only grid_pos provided: probability 0.0363, prediction 'no podium', 27 features imputed

Unknown feature name -> Unknown feature(s): ['grid_position']. Valid features: ['gri ...
```

	features provided	probability	prediction
record			
complete	28	0.7519	podium
partially missing	22	0.7371	podium
grid position only	1	0.0363	no podium



What the demonstration shows. The function returns a prediction for a complete record, for a record with several missing features, and even for a record with a single feature. As features are removed, the missing ones are replaced by the training median — the value of a *typical mid-field entry* — so the prediction is pulled towards an average car. The output reports which features were imputed; a prediction with many imputed features should be trusted less. An unknown feature name is rejected instead of being ignored.

---

15. Limitations and summary

15.1 Known limitations

Data

- **Qualifying coverage drift.** The `qualifying` table is empty before 1994 and patchy until 2002, so the model is built on, and valid for, 2003 onward only. Within 2003–2024 the qualifying *format* also changed (single-lap runs in 2003–2005, knockout sessions from 2006), so "gap to pole" does not mean exactly the same thing in every season.
- **Sprint weekends.** On sprint weekends in 2021–2022 the grid was set by the sprint, not by qualifying. `grid_pos` is still correct (it is the real starting slot) but `grid_drop` then mixes penalties with sprint results. Sprint results themselves are excluded because their position relative to qualifying changed between seasons.

- **Team identity.** Form and standings follow `constructorId`. When a team is renamed it receives a new id and its history restarts, although it is the same organisation.
- **Shared drives.** The rule for collapsing duplicate driver–race rows uses the outcome to choose the row. It affects only 85 pairs before 1979 and no row of the modeling table.
- **Status classification.** "Mechanical retirement" is built from free-text status labels and involves judgment for borderline labels; the sensitivity check in Question 3 shows how much this matters.
- **No information on weather, tyres, strategy, practice pace or upgrades**, all of which influence results and some of which are known before the race.

Modeling

- **Era sensitivity.** Regulations, points systems, field size and reliability all change over time. The test seasons belong to a new regulation era, and performance varies noticeably from season to season. A model trained through 2018 is being applied six seasons later; in real use it would be re-trained every season.
- **Small sample.** There are 900 podiums in training and about 200 in each of validation and test. Differences of one or two hundredths between models are within noise, as the bootstrap intervals show.
- **Rows are scored independently.** The models do not know that exactly three drivers per race are positive. The top-three-per-race rule (Section 11.1) uses this at decision time only.
- **Irreducible uncertainty.** First-lap incidents, failures, safety cars and rain cannot be seen in pre-race data. Case B in Section 13.3 is an example.
- **Validation reuse.** The validation seasons are used to choose hyperparameters, thresholds and the stopping epoch of the network, so validation scores are slightly optimistic. The test seasons are used once, at the end.
- **No re-fit on train + validation.** The final models are trained on 2003–2018 only. Re-fitting on 2003–2021 before predicting 2022–2024 would probably help, but would leave no data for early stopping, so it was not done.

Explainability

- **Explanations are not causal.** Importance means the model relies on a feature, not that the feature causes podiums.
- **Correlated features** share importance between them, so the ranking of similar features is unstable; group-level results are the reliable ones.

15.2 Verification that the raw files were not modified

```
Raw CSV files unchanged on disk: 14 of 14
In-memory RAW frames still identical to the files: True
```

15.3 Summary

- **Data.** 14 raw tables audited: all primary and foreign keys are sound; about 160,000 hidden missing values (`\\N` and empty strings) were converted; dates and durations were typed; 91 surplus shared-drive rows were collapsed so that `(raceId, driverId)` is unique.
- **Data-engineering answers.** (1) Front-row starts convert to podiums about 60% of the time over all seasons and more often in the hybrid era, and the circuit ranking is not stable across eras. (2) There is no home-race advantage once starting position is held fixed. (3) Mechanical retirements fell from the 2014–2021 era to 2022–2024, and the least reliable constructors differ between the two.
- **Modeling table.** 8,970 driver–race rows (2003–2024), 28 pre-race features in six groups, verified leakage-safe by a scramble-the-future test.

- **Models.** Selected on validation PR-AUC: **Gradient boosting**. On the unseen 2022–2024 seasons it reaches PR-AUC **0.691**, ROC-AUC **0.937** and F1 **0.667**, against PR-AUC 0.695 for the logistic baseline, 0.646 for the grid rule and about 0.15 for a random model.
- **Main finding.** Starting position and qualifying pace carry most of the predictable signal; team and driver form add a modest further gain; and the learned models are statistically hard to tell apart on 68 test races. What remains unpredicted is mostly what happens during the race.